Index

Practical No.	Topic	Date	Signature
	Introduction to AI and Intelligent Agents		
1.	Design a simple rational agent by defining PEAS for Vacuum Cleaner Environment and Autonomous Taxi. Classify environments (Fully / Partially Observable, Deterministic / Stochastic, Episodic / Sequential).		
	Problem Solving by Searching (Uninformed Search)		
2.	Given an initial configuration of the 8-puzzle and a goal configuration, write a Python program to find the shortest sequence of moves to reach the goal state using Breadth-First Search (BFS).		
3.	Given two water jugs of 4 litres and 3 litres capacity, write a Python program to obtain exactly 2 litres in one jug using Depth-First Search (DFS). [Vary capacity of jugs.]		
4.	Given a weighted graph representing cities and distances between them, write a Python program to find the least-cost path from Arad to Bucharest using Uniform Cost Search (UCS). [Provide any other weighted graph for applying UCS.]		
	Problem Solving by Searching (Informed Search)		
5.	Given an initial configuration of the 8-puzzle and a goal configuration, write a Python program to find the solution using Greedy Best-First Search with the Manhattan distance heuristic.		
6.	Given an initial configuration of the 8-puzzle and a goal configuration, write a Python program to find the shortest path using A* Search with the Manhattan distance heuristic.		
7.	Given a weighted graph representing cities and distances between them, write a Python program to find the shortest path from Arad to Bucharest using A* Search with a heuristic function (straight-line distance to destination).		

Practical No.	Topic	Date	Signature
	Bayes' Rule Application		
8.	Scenario: A medical test is used to detect a particular disease. The probability that a randomly selected person has the disease is 1%. If a person has the disease, the test returns positive with probability 99%. If a person does not have the disease, the test still returns positive with probability 5% (false positive rate). A person takes the test and receives a positive result. Problem Statement: Write a Python program to compute the probability that the person actually has the disease given that the test result is positive.		
	Naïve Bayes' Classification		
9.	Using a small dataset (e.g. weather or spam dataset), implement a Naïve Bayes classifier using scikit-learn.		
	Probability Reasoning (Bayesian Networks and Inference)		
10.	Using a simple example (Burglary-Alarm), create a Bayesian Network using pgmpy.		
11.	Write a program to compute posterior probability using Bayesian Networks (Burglar | Alarm = True).		
	Machine Learning		
12.	Using any small dataset (e.g. Iris or a CSV dataset), write a Python program to demonstrate the basic Machine Learning workflow: (a) Load dataset using Pandas. (b) Perform basic pre-processing (handling missing values or scaling). (c) Split dataset into training and testing sets. (d) Train a simple classifier. (e) Display training and testing accuracy.		
13.	Using a suitable regression dataset, implement Linear Regression using scikit-learn.		
14.	Using the MNIST dataset (Digit vs Not Digit), implement a binary classifier.		
15.	Implement K-Fold Cross-Validation on a classification model.		
16.	Implement a Decision Tree Classifier using the Iris or any relevant dataset.		
17.	Implement Decision Tree Regression on a regression dataset.		
18.	Implement K-Nearest Neighbors (kNN) classifier.		

Practical No.	Topic	Date	Signature
19.	Implement multiclass classification using Logistic Regression or Decision Tree.		

Jira Practicals

Practical No.	Topic	Date	Signature
1	Jira Cloud Setup & Project Creation		
2	Team Creation & Role Assignment in Jira Cloud		
3	Issue Types & Configuration		
4	Issue Management		
5	Workflow Customization		
6	Scrum Board & Sprint Management		

Practical 1

Aim: Design A Simple Rational Agent By Defining PEAS For Vacuum Cleaner Environment

Autonomous Taxi.

Classify Environments (Fully / Partially Observable , deterministic / Stochastic, Episodic / Sequential).Implement A Simple Table-Driven Agent In Python.

Introduction To AI and Intelligent Agents

CODE:

```
table = {
    ("A", "Dirty") : "Suck",
    ("A", "Clean") : "Move Right",
    ("B", "Dirty") : "Suck",
    ("B", "Clean") : "Move Left"
}
while True:
    location = input (" Enter location (A/B) : ").upper()
    status = input ("Enter status (Dirty/Clean):").capitalize()
    action = table.get ((location, status),"NoOp")
    print ("Action =",action)
    choice = input("Do you want to continue? (yes/no):").lower()
    if choice !="yes":
        break
```

OUTPUT:

```
Enter location (A/B) : A
Enter status (Dirty/Clean):Dirty
Action = Suck
Do you want to continue? (yes/no):no
```

Practical 2

Aim: Given An Initial Configuration Of The 8-Puzzle And A Goal Configuration Write A Python Program to Find The Shortest Sequence Of Moves To Reach The GoalState Using Breadth-First Search(BFS).

Problem Solving By Searching(Uninformed Search)

CODE:

```
from collections import deque
#Goal state
GOAL=(1,2,3,
      4,5,6,
      7,8,0)
#posiible move up,down,left,right
MOVES=[(-1,0),(1,0),(0,-1),(0,1)]
def get_neighbors(state):
    neighbors=[]
    zero_index=state.index(0)
    row,col=divmod(zero_index,3)
    for dr,dc in MOVES:
        new_row,new_col=row+dr,col+dc
        if 0<=new_row<3 and 0<=new_col<3:
            new_index=new_row*3+new_col
            new_state=list(state)
            new_state[zero_index], new_state[new_index]=(
                new_state[new_index],
                new_state[zero_index],
            )
            neighbors.append(tuple(new_state))
    return neighbors
```

```
def bfs(start):
    queue=deque([(start,[])])
    visited={start}
    while queue:
        current,path=queue.popleft()
        if current==GOAL:
            return path+[current]
        for neighbor in get_neighbors(current):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor,path+[current]))
    return None
def print_state(state):
    for i in range(0,9,3):
        print(state[i:i+3])
    print()
start_state=(
    1,2,3,
    4,8,5,
    7,6,0
)
solution=bfs(start_state)
if solution:
    print("Solution found In",len(solution) -1,"moves:\n")
    for step in solution:
        print_state(step)
else:
    print("No Solution exists.")
```

OUTPUT:

```
Solution found In 4 moves:

(1, 2, 3)
(4, 8, 5)
(7, 6, 0)

(1, 2, 3)
(4, 8, 5)
(7, 0, 6)

(1, 2, 3)
(4, 0, 5)
(7, 8, 6)

(1, 2, 3)
(4, 5, 0)
(7, 8, 6)

(1, 2, 3)
(4, 5, 6)
(7, 8, 0)
```

Practical 3

Aim: Given Two Water Jugs Of 4 Litres and 3 Litres Capacity , Write A Python Program To Obtain Exactly 2 Litres In One Jug Using Depth-First Search(DFS).[Vary Capacity Of Jugs].

Problem Solving By Searching(Uninformed Search)

CODE:

```
j1 = int(input("Enter capacity of Jug 1: "))
j2 = int(input("Enter capacity of Jug 2: "))
goal = int(input("Enter target amount: "))
visited = set()
def dfs(x, y, path):
    # Goal check
```

```
    if x == goal or y == goal:
        path.append((x, y))
        print("\nGoal Achieved Successfully!")
        print("Solution Path:")
        for state in path:
            print(state)
        return True
    # Check if state is already visited
    if (x, y) in visited:
        return False
    visited.add((x, y))
    path.append((x, y))
    # Generate possible next states
    next_states = [
        (j1, y),   # Fill Jug 1
        (x, j2),   # Fill Jug 2
        (0, y),    # Empty Jug 1
        (x, 0),    # Empty Jug 2
        # Pour Jug 1 -> Jug 2
        (x - min(x, j2 - y),
         y + min(x, j2 - y)),
        # Pour Jug 2 -> Jug 1
        (x + min(y, j1 - x),
         y - min(y, j1 - x))
    ]
    for state in next_states:
        if dfs(state[0], state[1], path.copy()):
            return True
    return False
# Start DFS
if not dfs(0, 0, []):
    print("\nNo solution found.")
```

OUTPUT:

```
Enter capacity of Jug 1: 4
Enter capacity of Jug 2: 3
Enter target amount: 2
Goal Achieved Successfully!
Solution Path:
(0, 0)
(4, 0)
(4, 3)
(0, 3)
(3, 0)
(3, 3)
(4, 2)
```

Practical 4

Aim: Given a weighted graph representing cities and distances between them, write a Python program to find the least cost path from Arad to Bucharest using Uniform Cost Search (UCS).

[Provide any other weighted graph for applying UCS.]

Problem Solving by Searching (Uninformed Search)

CODE:

```
import heapq
graph = {
    'Mumbai': [('Pune', 150), ('Hyderabad', 710)],
    'Pune': [('Mumbai', 150), ('Bangalore', 840)],
    'Hyderabad': [('Mumbai', 710), ('Bangalore', 570)],
    'Bangalore': []
}
def ucs(start, goal):
    queue = [(0, start, [start])]
    visited = []
```

```
    while queue:
        cost, city, path = heapq.heappop(queue)
        if city == goal:
            return cost, path
        if city not in visited:
            visited.append(city)
            for neighbor, distance in graph[city]:
                heapq.heappush(
                    queue,
                    (cost + distance, neighbor, path + [neighbor])
                )
cost, path = ucs("Mumbai", "Bangalore")
print("Least-cost path:")
print("Path:", " -> ".join(path))
print("Total Distance:", cost, "km")
```

OUTPUT:

```
Least-cost path:
Path: Mumbai -> Pune -> Bangalore
Total Distance: 990 km
```

Practical 5

Aim: Given AN Initial Configuration Of the 8-Puzzle And A Goal Configuration, Write A Python Program To Find The Solution Using Greedy Best-First Search with the Manhattan Distance Heuristic.

Problem Solving by Searching (Informed Search)

CODE:

```
import heapq
# Display puzzle state
def print_state(state):
    for i in range(0, 9, 3):
        print(state[i], state[i+1], state[i+2])
    print()
# Manhattan Distance Heuristic
def heuristic(state, goal):
    distance = 0
    for num in range(1, 9):
        current = state.index(num)
        target = goal.index(num)
        x1, y1 = current // 3, current % 3
        x2, y2 = target // 3, target % 3
        distance += abs(x1 - x2) + abs(y1 - y2)
    return distance
# Generate next states
def get_neighbors(state):
    neighbors = []
    blank = state.index(0)
    moves = [-3, 3, -1, 1]
    for move in moves:
        new = blank + move
        if new >= 0 and new < 9:
            if blank % 3 == 0 and move == -1:
                continue
            if blank % 3 == 2 and move == 1:
                continue
            temp = list(state)
            temp[blank], temp[new] = temp[new], temp[blank]
            neighbors.append(tuple(temp))
    return neighbors
# Greedy Best First Search
def greedy(initial, goal):
    queue = []
    heapq.heappush(
        queue,
```

```
        (heuristic(initial, goal), initial, [])
    )
    visited = set()
    while queue:
        h, state, path = heapq.heappop(queue)
        if state == goal:
            return path + [state]
        if state not in visited:
            visited.add(state)
            for next_state in get_neighbors(state):
                heapq.heappush(
                    queue,
                    (
                        heuristic(next_state, goal),
                        next_state,
                        path + [state]
                    )
                )
    return None
# Initial and Goal State
initial = (
    1, 2, 3,
    4, 0, 6,
    7, 5, 8
)
goal = (
    1, 2, 3,
    4, 5, 6,
    7, 8, 0
)
print("Initial State:")
print_state(initial)
print("Goal State:")
print_state(goal)
solution = greedy(initial, goal)
print("Solution:")
for step, state in enumerate(solution):
    print("Step", step)
    print_state(state)
print("Goal reached successfully.")
```

OUTPUT:

```
Initial State:
1 2 3
4 0 6
7 5 8

Goal State:
1 2 3
4 5 6
7 8 0

Solution:
Step 0
1 2 3
4 0 6
7 5 8

Step 1
1 2 3
4 5 6
7 0 8

Step 2
1 2 3
4 5 6
7 8 0

Goal reached successfully.
```

Practical 6

Aim: Given An INITIAL Configuration Of The 8-Puzzle And A Goal Configuration, Write A Python Program To Find The Shortest Path Using A* Search With The Manhattan Distance Heuristics.

Problem Solving by Searching (Informed Search)

CODE:

```
import heapq
# Display puzzle state
def print_state(state):
    for i in range(0, 9, 3):
        print(state[i], state[i+1], state[i+2])
    print()
# Manhattan Distance Heuristic
def heuristic(state, goal):
    distance = 0
    for num in range(1, 9):
        current = state.index(num)
        target = goal.index(num)
        x1, y1 = current // 3, current % 3
        x2, y2 = target // 3, target % 3
        distance += abs(x1 - x2) + abs(y1 - y2)
    return distance
# Generate next states
def get_neighbors(state):
    neighbors = []
    blank = state.index(0)
    moves = [-3, 3, -1, 1]
    for move in moves:
        new = blank + move
        if new >= 0 and new < 9:
            if blank % 3 == 0 and move == -1:
                continue
            if blank % 3 == 2 and move == 1:
                continue
            temp = list(state)
            temp[blank], temp[new] = temp[new], temp[blank]
            neighbors.append(tuple(temp))
    return neighbors
# A* Search
def astar(initial, goal):
    queue = []
    # f(n) = g(n) + h(n)
    heapq.heappush(
        queue,
        (heuristic(initial, goal), 0, initial, [])
    )
    visited = set()
    while queue:
        f, g, state, path = heapq.heappop(queue)
        if state == goal:
            return path + [state]
        if state not in visited:
            visited.add(state)
            for next_state in get_neighbors(state):
                new_g = g + 1
                new_f = new_g + heuristic(next_state, goal)
                heapq.heappush(
                    queue,
                    (new_f, new_g, next_state, path + [state])
                )
    return None
# Initial and Goal State
initial = (
    1, 2, 3,
    4, 0, 6,
    7, 5, 8
```

```
)
goal = (
    1, 2, 3,
    4, 5, 6,
    7, 8, 0
)
print("Initial State:")
print_state(initial)
print("Goal State:")
print_state(goal)
solution = astar(initial, goal)
print("Shortest Path using A* Search:")
step = 1
for state in solution[1:]:
    print("Step", step)
    print_state(state)
    step += 1
print("Goal reached successfully.")
print("Total number of moves:", len(solution) - 1)
```

OUTPUT:

```
Initial State:
1 2 3
4 0 6
7 5 8

Goal State:
1 2 3
4 5 6
7 8 0

Shortest Path using A* Search:
Step 1
1 2 3
4 5 6
7 0 8

Step 2
1 2 3
4 5 6
7 8 0

Goal reached successfully.
Total number of moves: 2
```

Practical 7

Aim: Given a weighted graph representing cities and distances between them, write a Python program to find the shortest path from Arad to Bucharest using A Search with a heuristic function (straight-line distance to destination).*

Problem Solving by Searching (Informed Search)

CODE:

```
import heapq
# Graph: city and distance to connected cities
graph = {
    'Arad': [('Zerind',75), ('Sibiu',140), ('Timisoara',118)],
    'Zerind': [('Arad',75), ('Oradea',71)],
    'Oradea': [('Zerind',71), ('Sibiu',151)],
    'Sibiu': [('Arad',140), ('Oradea',151), ('Fagaras',99), ('Rimnicu Vilcea',80)],
    'Timisoara': [('Arad',118), ('Lugoj',111)],
    'Lugoj': [('Timisoara',111), ('Mehadia',70)],
    'Mehadia': [('Lugoj',70), ('Drobeta',75)],
    'Drobeta': [('Mehadia',75), ('Craiova',120)],
    'Craiova': [('Drobeta',120), ('Rimnicu Vilcea',146), ('Pitesti',138)],
```

```
    'Rimnicu Vilcea': [('Sibiu',80), ('Craiova',146), ('Pitesti',97)],
    'Fagaras': [('Sibiu',99), ('Bucharest',211)],
    'Pitesti': [('Rimnicu Vilcea',97), ('Craiova',138), ('Bucharest',101)],
    'Bucharest': []
}
# Heuristic: straight-line distance to Bucharest
heuristic = {
    'Arad': 366,
    'Zerind': 374,
    'Oradea': 380,
    'Sibiu': 253,
    'Timisoara': 329,
    'Lugoj': 244,
    'Mehadia': 241,
    'Drobeta': 242,
    'Craiova': 160,
    'Rimnicu Vilcea': 193,
    'Fagaras': 176,
    'Pitesti': 100,
    'Bucharest': 0
}
def astar(start, goal):
    queue = []
    # queue contains: f_cost, g_cost, city, path
    heapq.heappush(queue, (heuristic[start], 0, start, [start]))
    visited = set()
    while queue:
        f_cost, g_cost, city, path = heapq.heappop(queue)
        if city == goal:
            return g_cost, path
        if city not in visited:
            visited.add(city)
            for neighbor, distance in graph[city]:
                new_g = g_cost + distance
                new_f = new_g + heuristic[neighbor]
                heapq.heappush(
                    queue,
                    (new_f, new_g, neighbor, path + [neighbor])
                )
    return None, []
cost, path = astar('Arad', 'Bucharest')
print("Shortest path using A* Search:")
print("Path:", " -> ".join(path))
print("Total Cost:", cost)
```

OUTPUT:

```
Path: Arad -> Sibiu -> Rimnicu Vilcea -> Pitesti -> Bucharest
Total Cost: 418
```

Practical 8

Aim: Given Scenario

Problem Statement:

Write A Python Program To Compute The Probability that the person actually has the test result is positive.

Bayes' Rule Application

CODE:

```
# Probability of having disease
P_D = 0.01
# Probability of positive test if disease exists
P_Pos_D = 0.99
# Probability of positive test if disease does not exist
P_Pos_NotD = 0.05
# Probability of not having disease
```

```
P_NotD = 1 - P_D
# Bayes Rule
P_D_Pos = (P_Pos_D * P_D) / ((P_Pos_D * P_D) + (P_Pos_NotD * P_NotD))
print("Probability that person actually has disease:")
print(round(P_D_Pos * 100, 2), "%")
```

OUTPUT:

```
Probability that person actually has disease:
16.67 %
```

Practical 9

Aim: Using a small dataset (e.g., Weather or Spam dataset), implement a Naive Bayes classifier using Scikit-Learn.

Naive Bayes' Classification

CODE:

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.naive_bayes import GaussianNB
# Dataset
data = {
    'Outlook': ['Sunny', 'Sunny', 'Overcast', 'Rainy', 'Rainy', 'Rainy', 'Overcast'],
    'Temperature': ['Hot', 'Hot', 'Hot', 'Mild', 'Cool', 'Cool', 'Cool'],
    'Humidity': ['High', 'High', 'High', 'High', 'Normal', 'Normal', 'Normal'],
    'Windy': ['False', 'True', 'False', 'False', 'False', 'True', 'True'],
    'Play': ['No', 'No', 'Yes', 'Yes', 'Yes', 'No', 'Yes']
}
df = pd.DataFrame(data)
# Label Encoding
le = LabelEncoder()
for col in df.columns:
    df[col] = le.fit_transform(df[col])
# Input and output
X = df[['Outlook', 'Temperature', 'Humidity', 'Windy']]
y = df['Play']
# Create and train model
model = GaussianNB()
model.fit(X, y)
# Test data with column names
test = pd.DataFrame(
    [[2, 0, 0, 1]],
    columns=['Outlook', 'Temperature', 'Humidity', 'Windy']
)
# Prediction
prediction = model.predict(test)
if prediction[0] == 1:
    print("Prediction: Yes, we can play")
else:
    print("Prediction: No, we cannot play")
```

OUTPUT:

```
Prediction: No, we cannot play
```

Practical 10

Aim: Using A Simple Example (BurGlary-ALARM), Create A Bayesian Network using PGMPY.

Probability Reasoning(Bayesian Network And Inference)

CODE:

```
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination

model=DiscreteBayesianNetwork([
    ("Burglary","Alarm"),
    ("Earthquake","Alarm"),
    ("Alarm","JohnCalls"),
    ("Alarm","MaryCalls")
])

cpd_burglary=TabularCPD(
    variable="Burglary",
    variable_card=2,
    values=[[0.999],[0.001]],
    state_names={"Burglary":["False","True"]}
)
cpd_earthquake=TabularCPD(
    variable="Earthquake",
    variable_card=2,
    values=[[0.998],[0.002]],
    state_names={"Earthquake":["False","True"]}
)
cpd_alarm=TabularCPD(
    variable="Alarm",
    variable_card=2,
    values=[
        [0.999,0.71,0.06,0.05],
        [0.001,0.29,0.94,0.95]
    ],
    evidence=["Burglary","Earthquake"],
    evidence_card=[2,2],
    state_names={
        "Alarm":["False","True"],
        "Burglary":["False","True"],
        "Earthquake":["False","True"]
    }
)
cpd_john=TabularCPD(
    variable="JohnCalls",
    variable_card=2,
    values=[
        [0.95,0.10],
        [0.05,0.90]
    ],
    evidence=["Alarm"],
    evidence_card=[2],
    state_names={
        "JohnCalls":["False","True"],
        "Alarm":["False","True"]
    }
)
cpd_mary=TabularCPD(
    variable="MaryCalls",
    variable_card=2,
    values=[
        [0.99,0.30],
        [0.01,0.70]
    ],
    evidence=["Alarm"],
    evidence_card=[2],
    state_names={
```

```
        "MaryCalls":["False","True"],
        "Alarm":["False","True"]
    }
)
model.add_cpds(
    cpd_burglary,
    cpd_earthquake,
    cpd_alarm,
    cpd_john,
    cpd_mary
)
print("Is Bayesian Network valid?",model.check_model())
inference=VariableElimination(model)
result=inference.query(
    variables=["Burglary"],
    evidence={
        "JohnCalls":"True",
        "MaryCalls":"True"
    }
)
print("\nProbability of Burglary given JohnCalls=True and MarryCalls=True:")
print(result)
```

OUTPUT:

```
Is Bayesian Network valid? True

Probability of Burglary given JohnCalls=True and MarryCalls=True:
+-----------------+-----------------+
| Burglary        |   phi(Burglary) |
+=================+=================+
| Burglary(False) |          0.7158 |
+-----------------+-----------------+
| Burglary(True)  |          0.2842 |
+-----------------+-----------------+
```

Practical 11

Aim: Write A Program To Compute Posterior Probability Using Bayesian Networks (uBrglar |Alarm = True).

Probability Reasoning(Bayesian Network And Inference)

CODE:

```
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination

model = DiscreteBayesianNetwork([
    ("Burglary", "Alarm"), ("Earthquake", "Alarm")
])

cpd_burglary = TabularCPD(
    variable="Burglary",
    variable_card=2,
    values=[[0.999], [0.001]],
    state_names={"Burglary": ["False", "True"]}
)
cpd_earthquake = TabularCPD(
    variable="Earthquake",
    variable_card=2,
    values=[[0.998], [0.002]],
    state_names={"Earthquake": ["False", "True"]}
)
cpd_alarm = TabularCPD(
    variable="Alarm",
    variable_card=2,
```

```
    values=[
        [0.999, 0.71, 0.06, 0.05],  # Alarm = False
        [0.001, 0.29, 0.94, 0.95]   # Alarm = True
    ],
    evidence=["Burglary", "Earthquake"],
    evidence_card=[2, 2],
    state_names={
        "Alarm": ["False", "True"],
        "Burglary": ["False", "True"],
        "Earthquake": ["False", "True"]
    }
)
model.add_cpds(
    cpd_burglary,
    cpd_earthquake,
    cpd_alarm
)
print("Is Bayesian Network valid?", model.check_model())
inference = VariableElimination(model)

result = inference.query(
    variables=["Burglary"],
    evidence={"Alarm": "True"}
)
print("\nPosterior Probability:")
print("P(Burglary | Alarm = True)")
print(result)
```

OUTPUT:

```
Is Bayesian Network valid? True

Probability Probability:
P(Burglary|Alarm=True)
+-----------------+-----------------+
| Burglary        |   phi(Burglary) |
+=================+=================+
| Burglary(False) |          0.6264 |
+-----------------+-----------------+
| Burglary(True)  |          0.3736 |
+-----------------+-----------------+
```

Practical 12.1

Aim: Using Any Small Dataset (E.G.,Iris or A CSV Dataset), Write A Python Program To Demonstrate The Basic Machine Learning Workflow.

Machine Learning

CODE:

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# a) Load dataset using Pandas
data = pd.read_csv("student.csv")

print("Dataset:")
print(data)

# Convert Pass/Fail into numbers
le = LabelEncoder()
data["Result"] = le.fit_transform(data["Result"])
```

```
# Input and Output
X = data[["Hours", "Attendance"]]
y = data["Result"]

# b) Preprocessing (Scaling)
scaler = StandardScaler()
X = scaler.fit_transform(X)

# c) Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=1
)

# d) Train classifier
model = KNeighborsClassifier()
model.fit(X_train, y_train)

# e) Training and Testing Accuracy
train_pred = model.predict(X_train)
test_pred = model.predict(X_test)

print("\nTraining Accuracy:", accuracy_score(y_train, train_pred)*100)

print("Testing Accuracy:", accuracy_score(y_test, test_pred)*100)
```

OUTPUT:

```
Dataset:
   Hours  Attendance Result
0      2          60   Fail
1      3          65   Fail
2      4          70   Pass
3      5          75   Pass
4      6          80   Pass
5      1          50   Fail
6      7          90   Pass
7      3          55   Fail
8      8          95   Pass
9      5          85   Pass

Training Accuracy: 85.71428571428571
Testing Accuracy: 66.66666666666666
```

Practical 12.2

Aim: Using A Suitable Regression Dataset, Implement Linear Regression Using Scikit-Learn.

Machine Learning

CODE:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

# Load dataset
data = pd.read_csv("student_marks.csv")

# Input and output
X = data[["Hours"]]
y = data["Marks"]

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
```

```
)

# Train model
model = LinearRegression()
model.fit(X_train, y_train)

# Display accuracy
print("Training Accuracy:", round(model.score(X_train, y_train)*100,2))
print("Testing Accuracy:", round(model.score(X_test, y_test)*100,2))

# Predict marks for 11 study hours
new_data = pd.DataFrame([[11]], columns=["Hours"])
pred = model.predict(new_data)

print("Predicted Marks for 11 hours:", round(pred[0],2))
```

OUTPUT:

```
Training Accuracy: 99.64
Testing Accuracy: 98.71
Predicted Marks for 11 hours: 93.57
```

Practical 12.3

Aim: Using The MNIST Dataset(Digit Vs Not Digit), Implement A Binary Classifier.

Machine Learning

CODE:

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

# Load MNIST-like digits dataset
digits = load_digits()

X = digits.data
y = digits.target

# Binary classification
# True if digit is 5, otherwise False
y = (y == 5)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)

# Train model
model = LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)

# Accuracy
print("Training Accuracy:", model.score(X_train,y_train)*100)

print("Testing Accuracy:", model.score(X_test,y_test)*100)

# Predict first test sample
prediction = model.predict([X_test[0]])

if prediction[0]:
    print("Digit is 5")
else:
```

```
    print("Digit is not 5")
```

OUTPUT:

```
Training Accuracy: 100.0
Testing Accuracy: 98.88888888888889
Digit is not 5
```

Practical 12.4

Aim: Implement K-Fold Cross-Validation On A Classification Model.

Machine Learning

CODE:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsClassifier

# Load dataset
iris = load_iris()

X = iris.data
y = iris.target

# Create classification model
model = KNeighborsClassifier()

# Apply 5-fold cross-validation
scores = cross_val_score(model,X,y,cv=5)

print("Accuracy scores for each fold:")
print(scores)

print("Average Accuracy:", round(scores.mean() * 100, 2), "%")
```

OUTPUT:

```
Accuracy scores for each fold:
[0.96666667 1.         0.93333333 0.96666667 1.        ]

Average Accuracy: 97.33 %
```

Practical 12.5

Aim: Implement A Decision Tree Classifier Using The Iris Or Any Relevant Dataset.

Machine Learning

CODE:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
```

```
    test_size=0.3,
    random_state=1
)

# Create model
model = DecisionTreeClassifier()

# Train model
model.fit(X_train,y_train)

# Display accuracy
print("Training Accuracy:", model.score(X_train, y_train)*100)
print("Testing Accuracy:", model.score(X_test, y_test)*100)

# Prediction
prediction = model.predict([X_test[0]])
print("Predicted Class:", prediction[0])
```

OUTPUT:

```
Training Accuracy: 100.0
Testing Accuracy: 95.55555555555556
Predicted Class: 0
```

Practical 12.6

Aim: Implement Decision Tree Regression On A Regression Dataset

Machine Learning

CODE:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor

# Load dataset
data = pd.read_csv("student_marks.csv")

# Input and output
X = data[["Hours"]]
y = data["Marks"]

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)

# Create model
model = DecisionTreeRegressor()

# Train model
model.fit(X_train, y_train)

# Display accuracy
print("Training Accuracy:", model.score(X_train,y_train)*100)
print("Testing Accuracy:", model.score(X_test,y_test)*100)

# Prediction
new_data = pd.DataFrame([[11]], columns=["Hours"])
prediction = model.predict(new_data)
print("Predicted Marks for 11 Hours:",prediction[0])
```

OUTPUT:

```
Training Accuracy: 100.0
```

```
Testing Accuracy: 80.21978021978022
Predicted Marks for 11 Hours: 80.0
```

Practical 12.7

Aim: Implement K-Nearest Neighbors(KNN) Classifier.

Machine Learning

CODE:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

# Load dataset
iris = load_iris()

X = iris.data
y = iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)

# Create KNN model
model = KNeighborsClassifier(n_neighbors=3)

# Train model
model.fit(X_train, y_train)

# Display accuracy
print("Training Accuracy:", model.score(X_train, y_train)*100)
print("Testing Accuracy:", model.score(X_test, y_test)*100)

# Prediction
prediction = model.predict([X_test[0]])
print("Predicted Class:", prediction[0])
```

OUTPUT:

```
Training Accuracy: 95.23809523809523
Testing Accuracy: 97.77777777777777
Predicted Class: 0
```

Practical 12.8

Aim: Implement Multiclass Classification using Logistic Regression or Decision Tree

Machine Learning

CODE:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

# Load Iris dataset
iris = load_iris()
X = iris.data
y = iris.target
```

```
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,y,
    test_size=0.3,
    random_state=1
)

# Create Decision Tree model
model = DecisionTreeClassifier()

# Train model
model.fit(X_train,y_train)

# Display accuracy
print("Training Accuracy:",model.score(X_train,y_train)*100)
print("Testing Accuracy:",model.score(X_test,y_test)*100)
```

OUTPUT:

```
Training Accuracy: 100.0
Testing Accura
```

JIRA PRACTICAL

PRACTICAL 1

Title: Jira Cloud Setup & Project Creation

Project: Online Shopping System (Key: OSS) Date: _____ Sign: _____

Aim

To set up an Atlassian Jira Cloud account, navigate the primary dashboard interface, and create and configure a Company-managed Agile project (Scrum or Kanban).

Requirements

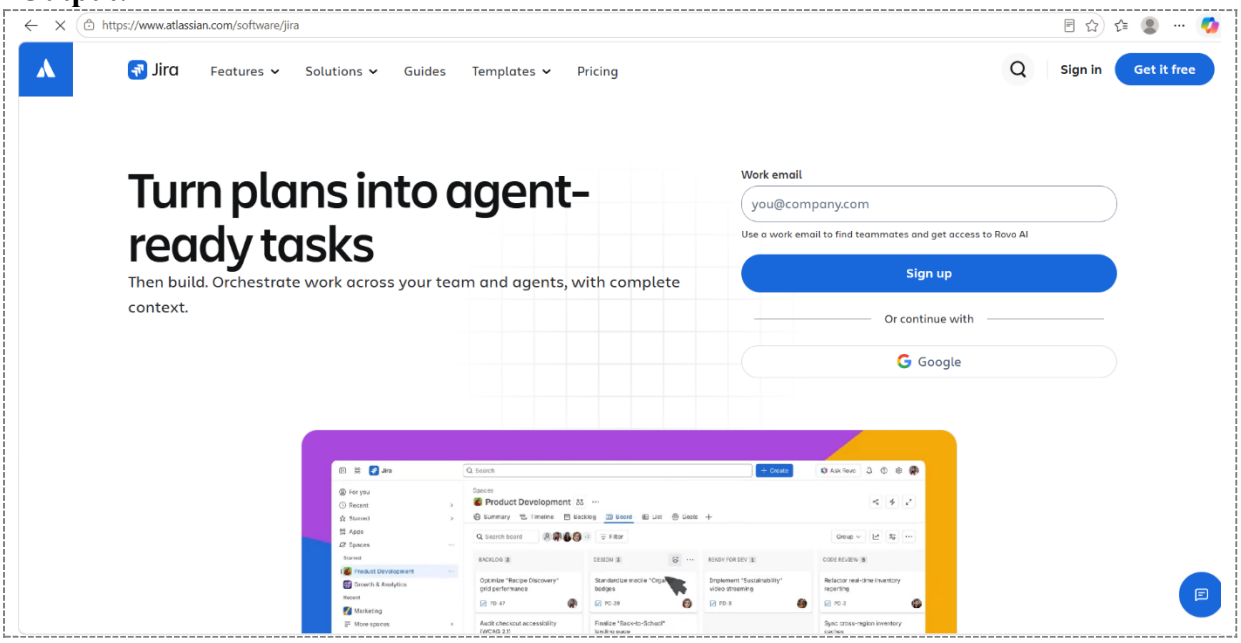
Computer/laptop with an active internet connection; web browser (Google Chrome, Microsoft Edge, Mozilla Firefox or Safari); valid personal or institutional email address.

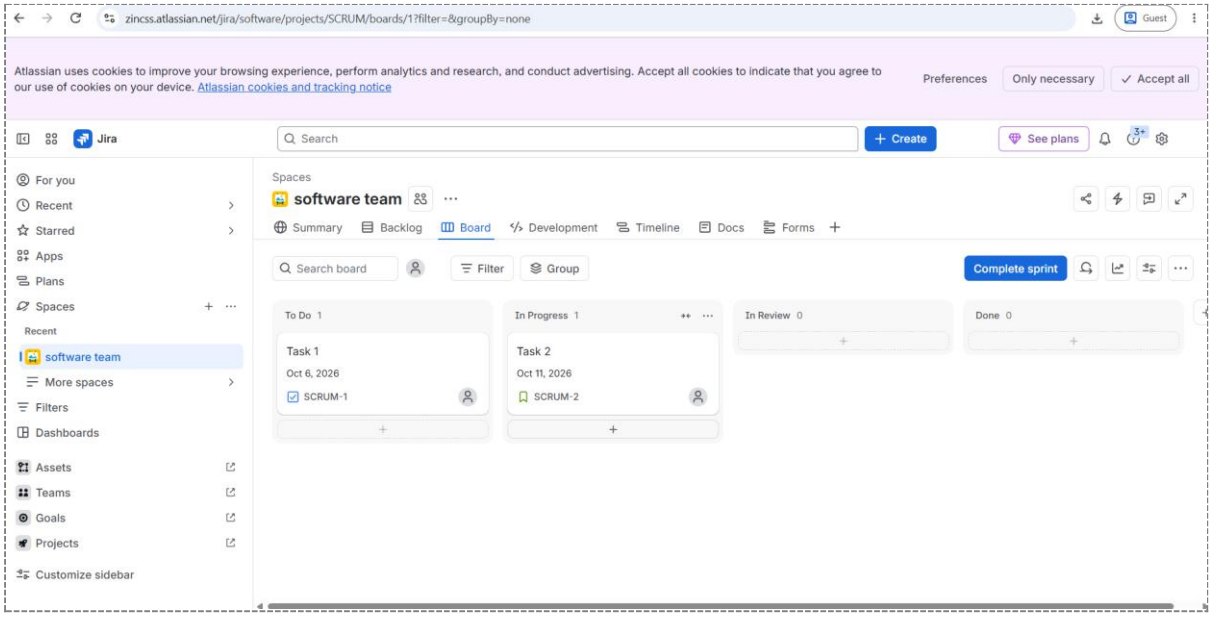
Procedure

Part A: Jira Cloud Account Creation & Interface Navigation

1. Open the browser and navigate to atlassian.com/software/jira.
2. Click "Get it free" or "Try Jira Software".
3. Sign up using a valid email address (or Google/Microsoft SSO).
4. Check the email inbox for the verification link from Atlassian and complete account activation.
5. Enter a unique site name when prompted (e.g. yourname-lab.atlassian.net).
6. Select Jira Software as the primary product.
7. Complete the onboarding questionnaire (or select Skip).
8. Once logged in, explore the navigation bar: Projects/Spaces, Filters/Search, Dashboards, Teams, Settings (gear icon) and User Profile/Avatar.

Output:



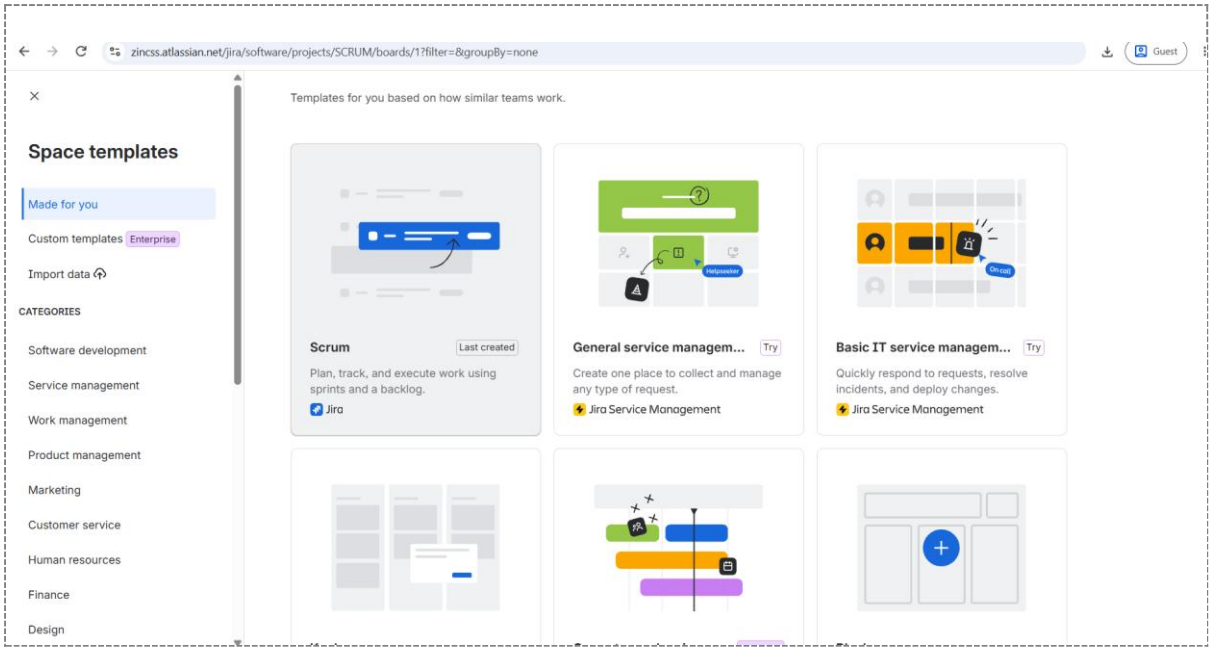


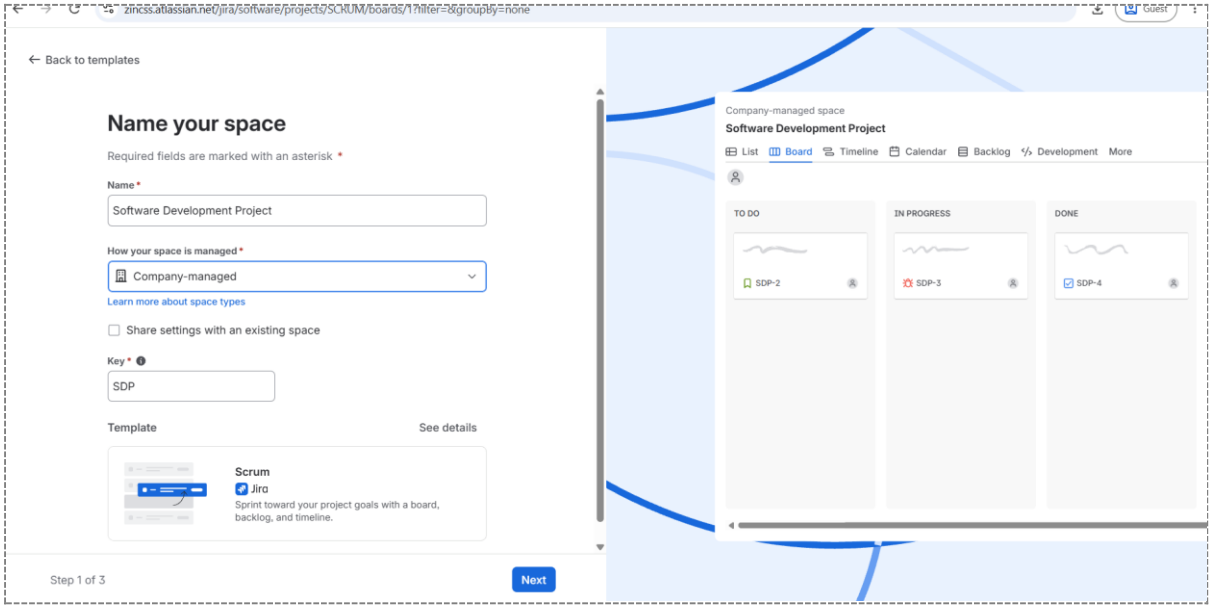
Part B: Creating a Company-managed Software Project

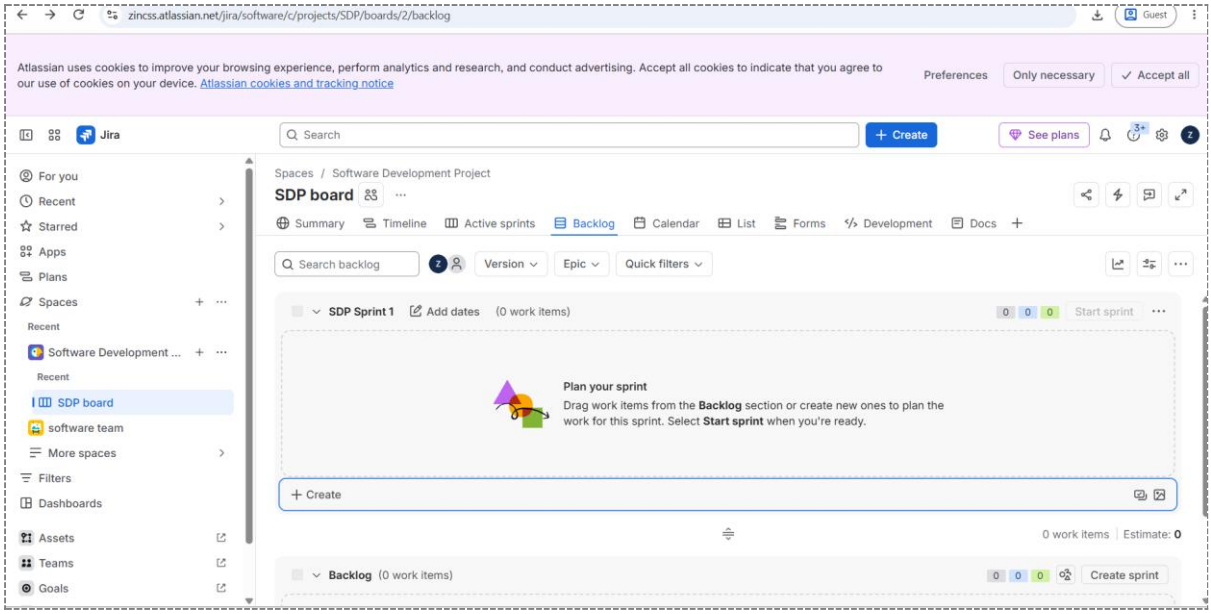
1. From the top navigation bar select Projects (or Spaces).
2. Click "Create project" (or "Create space").
3. Under the Software Development category select the Scrum (or Kanban) template.
4. When prompted for the project type, select Company-managed (do NOT select Team-managed).
5. Enter Project Name: Software Development Project; Key: SDP (auto-generated 2–10 character uppercase prefix, e.g. SDP-1, SDP-2).

6. Confirm Template = Scrum/Kanban and Type = Company-managed, then click "Create project".

Output:



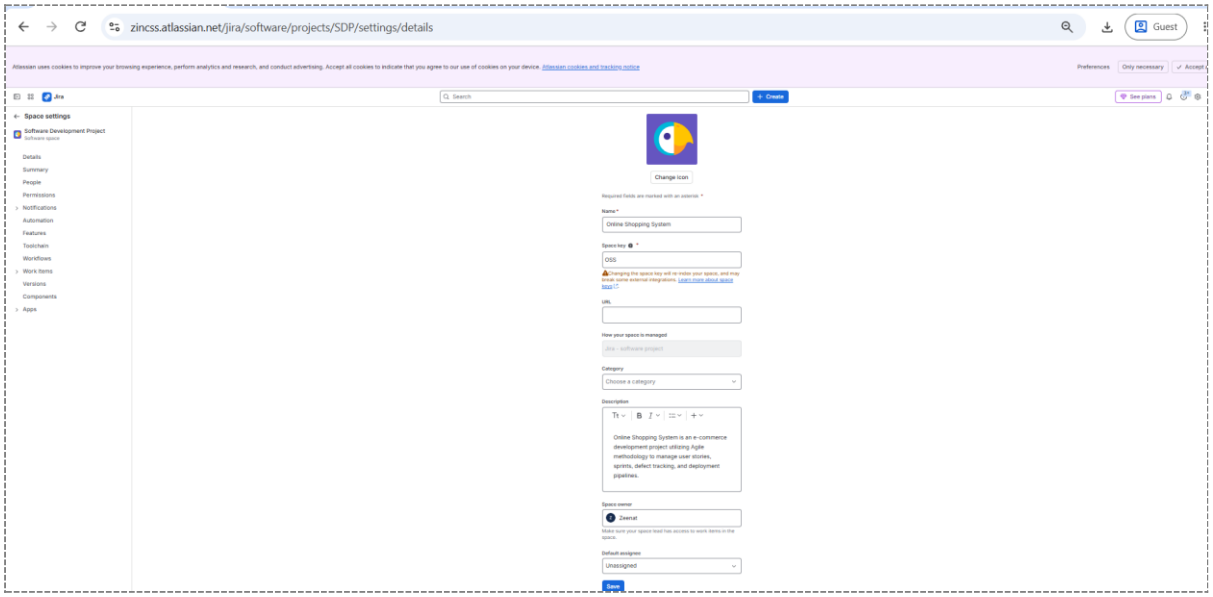


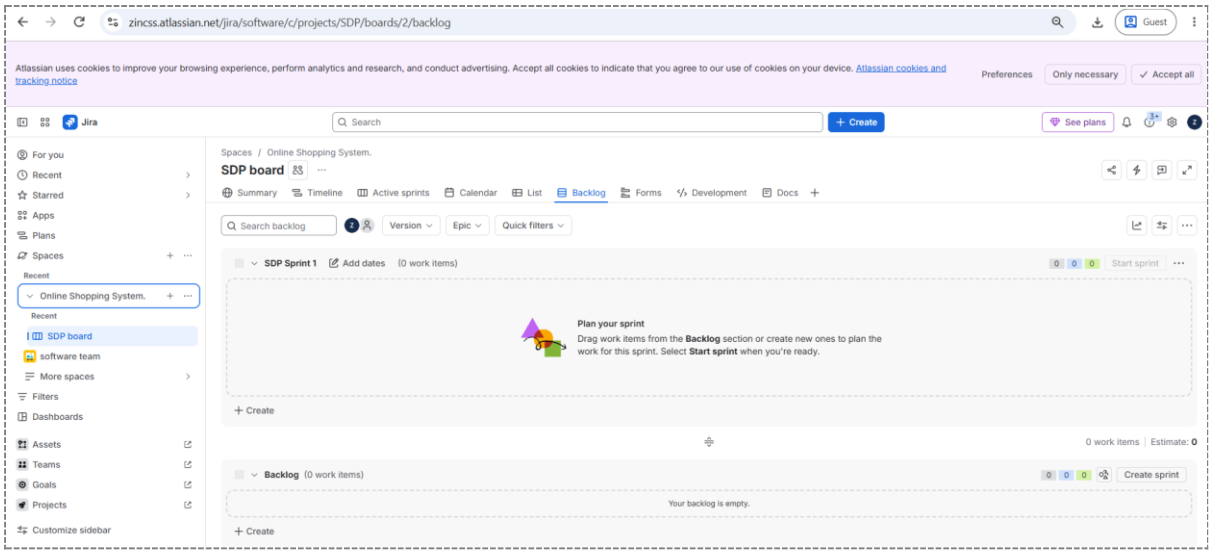


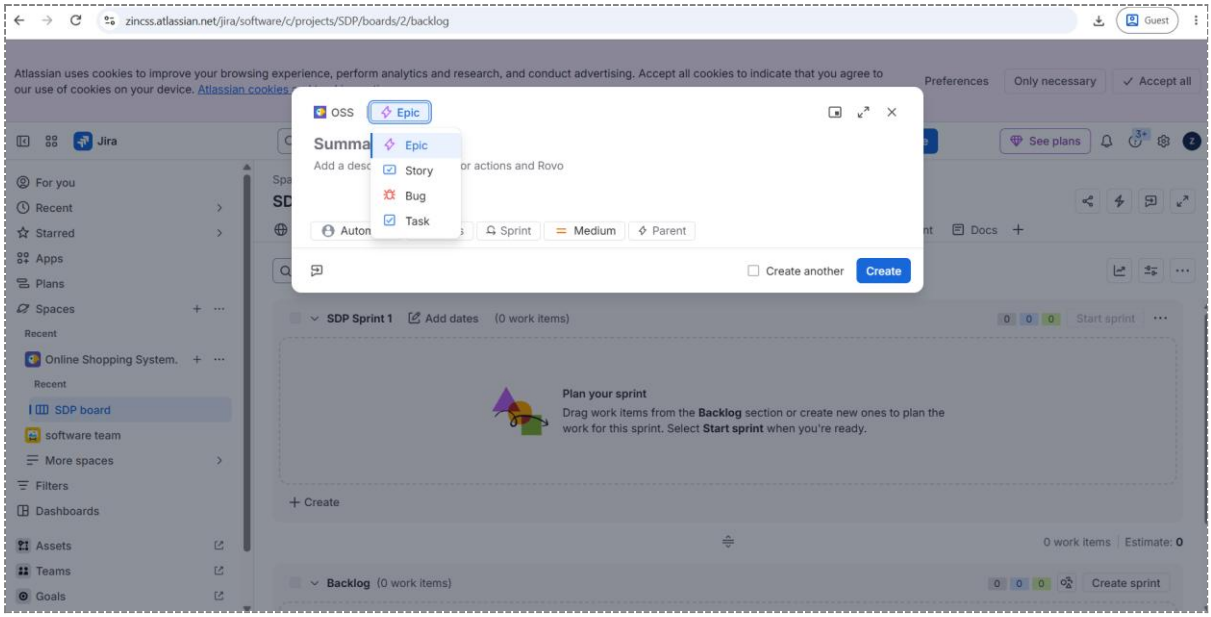
Part C: Configuring Project Details & Workspace Verification

1. Navigate to the newly created project.
2. In the left sidebar click Project settings (or Space settings) → Details.
3. Update Name: Online Shopping System.
4. Update Key: OSS (changing the key updates existing item links, so set it early).
5. Enter Description: "Online Shopping System is an e-commerce development project utilizing Agile methodology to manage user stories, sprints, defect tracking, and deployment pipelines."
6. Click "Save changes".
7. Verify the board: for Scrum check Backlog and Active Sprints; for Kanban check the columns To Do → In Progress → Done.
8. Click "+ Create" in the top bar and verify the available issue types: Epic, Story, Task, Bug and Sub-task.

Output:


zincss.atlassian.net/jira/software/projects/SDP/settings/details
Guest
Space settings
Software Development Project
Details
Summary
People
Permissions
Notifications
Automation
Features
Toolchain
Workflows
Work items
Versions
Components
Apps
Change icon
Name *
Online Shopping System
Space key *
OSS
URL
How your space is managed
Jira - software project
Category
Choose a category
Description
Online Shopping System is an e-commerce development project utilizing Agile methodology to manage user stories, sprints, defect tracking, and deployment pipelines.
Space owner
Default assignee
Unassigned
Save


zincss.atlassian.net/jira/software/c/projects/SDP/boards/2/backlog
Guest
Atlassian uses cookies to improve your browsing experience, perform analytics and research, and conduct advertising. Accept all cookies to indicate that you agree to our use of cookies on your device. Atlassian cookies and tracking notice
Preferences
Only necessary
Accept all
Jira
Search
Create
See plans
For you
Recent
Starred
Apps
Plans
Spaces
Recent
Online Shopping System.
Recent
SDP board
software team
More spaces
Filters
Dashboards
Assets
Teams
Goals
Projects
Customize sidebar
Spaces / Online Shopping System.
SDP board
Summary
Timeline
Active sprints
Calendar
List
Backlog
Forms
Development
Docs
Search backlog
Version
Epic
Quick filters
SDP Sprint 1
Add dates
(0 work items)
Start sprint
Plan your sprint
Drag work items from the Backlog section or create new ones to plan the work for this sprint. Select Start sprint when you're ready.
Create
0 work items
Estimate: 0
Backlog
(0 work items)
Create sprint
Your backlog is empty.
Create



Result

A Jira Cloud account was created, the dashboard was explored, and a Company-managed Scrum/Kanban project named Online Shopping System (OSS) was created, configured and verified.

PRACTICAL 2

Title: Team Creation & Role Assignment in Jira Cloud

Project: Online Shopping System (Key: OSS) Date: _____ Sign: _____

Aim

To add team members to a Jira Cloud project and assign appropriate project roles and permissions.

Requirements

Jira Cloud account with Administrator rights; project Online Shopping System (OSS) from Practical 1; name or Atlassian email IDs of the team members.

Sample Team Configuration

Sr.	Team Member	Responsibility	Jira Role
1	Member 1	Project coordination / management	Administrator
2	Member 2	Software development	Developer
3	Member 3	Software development	Developer
4	Member 4	Testing	Developer
5	Member 5	Requirement review / observation	Viewer

Role and Permission Relationship (may differ per project)

Permission	Possible Role
Browse project	Viewer / Developer
Create work items	Developer
Edit work items	Developer
Assign work items	Developer
Manage project	Administrator

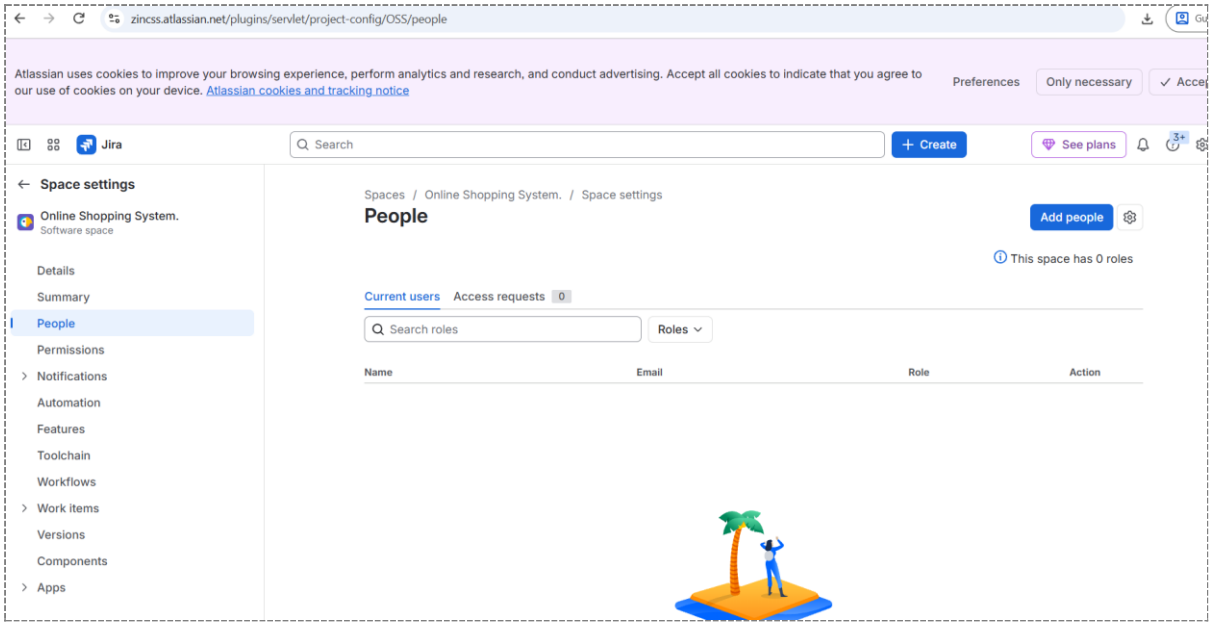
Procedure

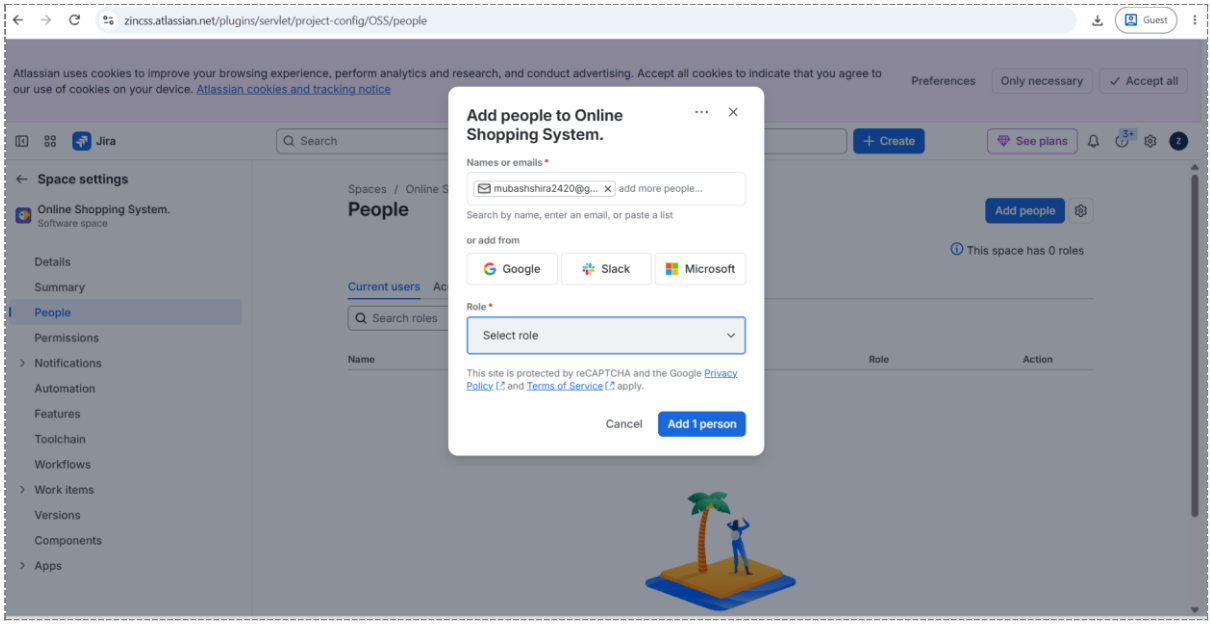
A. Add Team Members

1. Log in to Jira Cloud and open Spaces/Projects.
2. Select the Online Shopping System project created in Practical 1.
3. Open People from the project navigation (if not visible: Project/Space settings → People).
4. Click "Add people".
5. Enter the team member's name or Atlassian account email address.

6. Select the person from the search results (invite them to the Atlassian site first if they have no access).

Output:



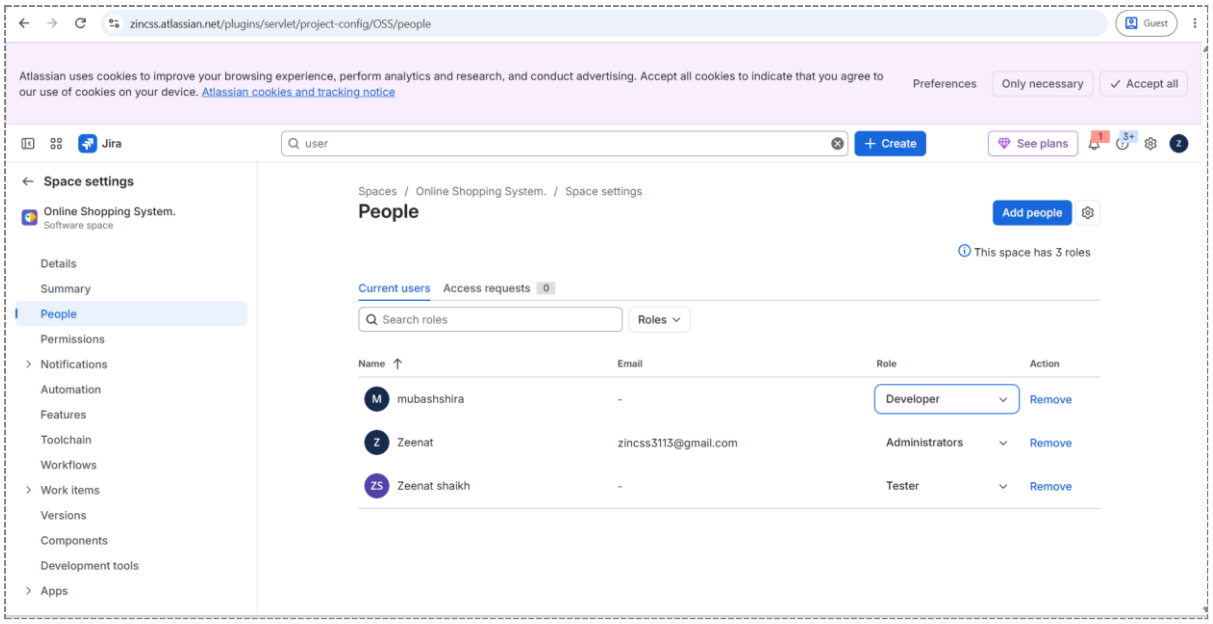


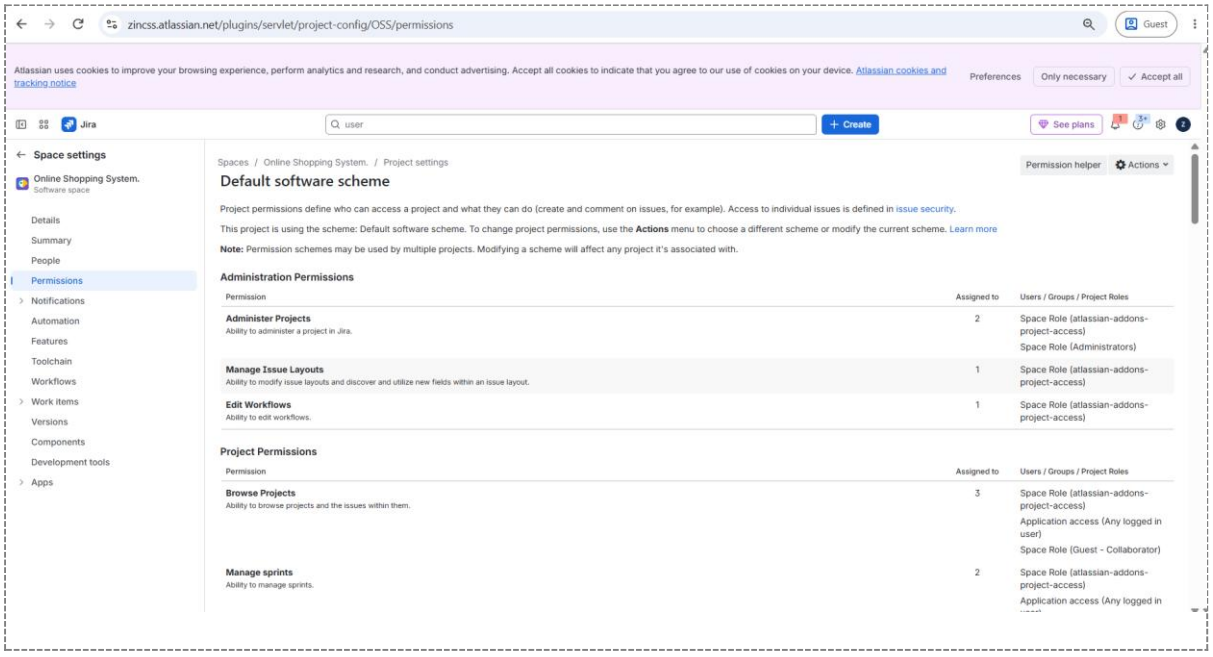
B. Assign Roles

1. Open Project/Space settings and locate People (Access/People).
2. After selecting the user, choose the project role (Administrator, Developer or Viewer) as per the sample team table.
3. Click "Add" / Save/apply the change.

4. Repeat for all team members (at least 4: Project Administrator, Developer 1, Developer 2, Viewer/Tester).
5. Return to the People page and verify that all users and their roles are displayed.
6. Open Project settings → Permissions and observe how roles are associated with permissions.

Output:





Result

Team members were successfully added to the Jira Cloud project, appropriate project roles were assigned and verified, and the association between roles and permissions was observed.

PRACTICAL 3

Title: Issue Types & Configuration

Project: Online Shopping System (Key: OSS) Date: _____ Sign: _____

Aim

To create and configure different Jira issue/work types such as Epic, Story, Task and Bug, and to create Components and Versions for a Company-managed software project.

Requirements

Jira Cloud with Administer Jira permission; Company-managed Scrum project Online Shopping System (OSS).

Work Items Created

Key	Type	Summary
OSS-1	Epic	User Authentication
OSS-2	Story	Customer Login
OSS-3	Task	Design Login Page
OSS-4	Bug	Login button does not respond

Components and Versions

Component	Purpose
Frontend	User interface and client-side functionality
Backend	Server-side application logic and API development
Database	Database design, queries and data management
Testing	Software testing, defect identification, verification and validation

Procedure

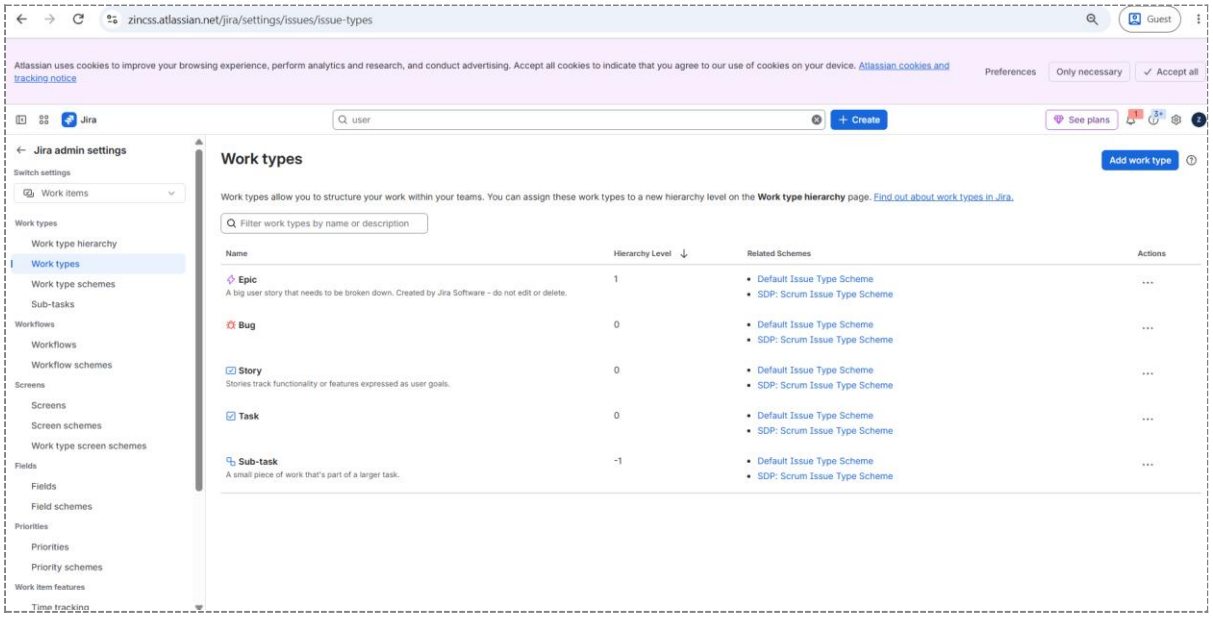
Part A: Create and Configure Issue Types

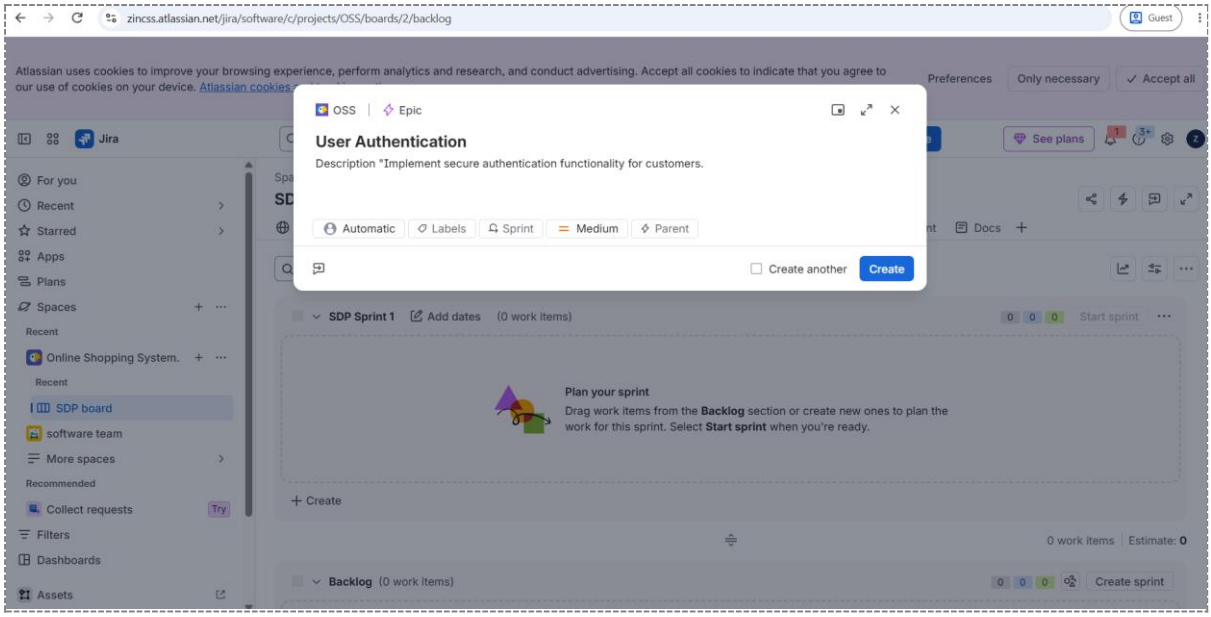
1. Open the Online Shopping System project and click Create.
2. Open the Work type/Issue type dropdown and verify Epic, Story, Task, Bug and Subtask are listed; cancel the dialog.
3. As Jira administrator click Settings (gear) → Work items → Work types and view the standard work types.
4. Locate Epic → More actions (•••) → Edit and review Name, Description and Icon. Description: "Represents a major feature or functional area of the Online Shopping

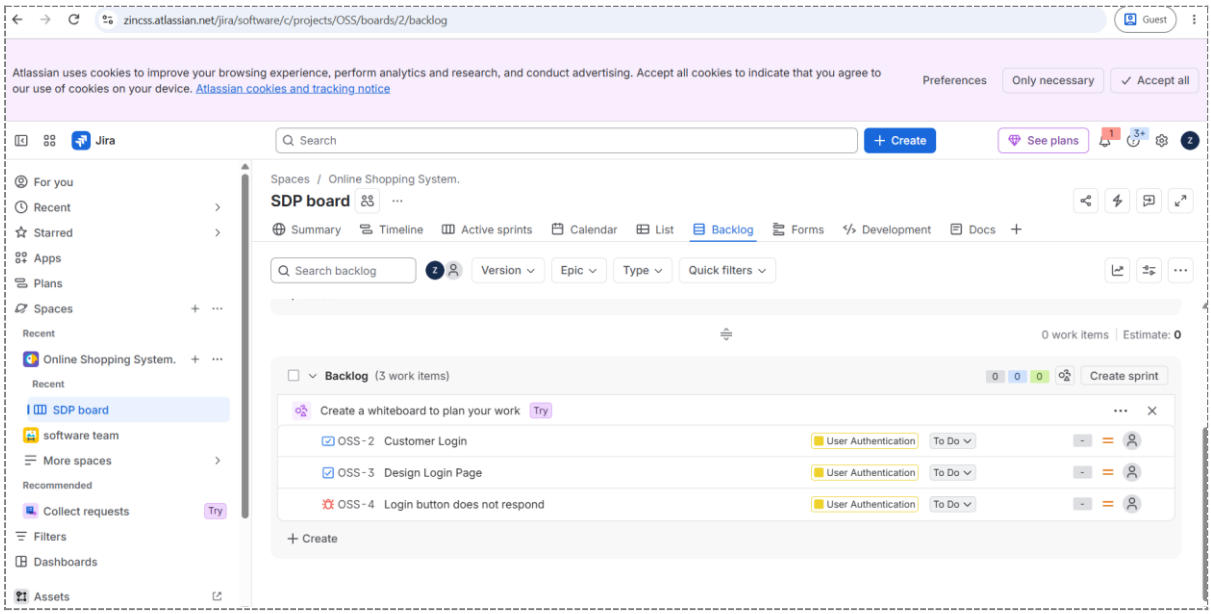
System that can be divided into multiple Stories, Tasks and Bugs." Save. (Do not rename standard types.)

5. Similarly review/edit Story ("Represents a user requirement or functionality that delivers value to the user."), Task ("Represents a specific activity that must be completed as part of the project.") and Bug ("Represents a defect or unexpected behavior in the software that needs to be investigated and fixed.").
6. Click Create → Epic: Summary "User Authentication", Description "Implement secure authentication functionality for customers." → Create (OSS-1).
7. Create Story "Customer Login" (As a registered customer, I want to log in using my email and password so that I can access my account), Task "Design Login Page" and Bug "Login button does not respond".
8. Open the Board/Backlog and verify the four created work items.

Output:





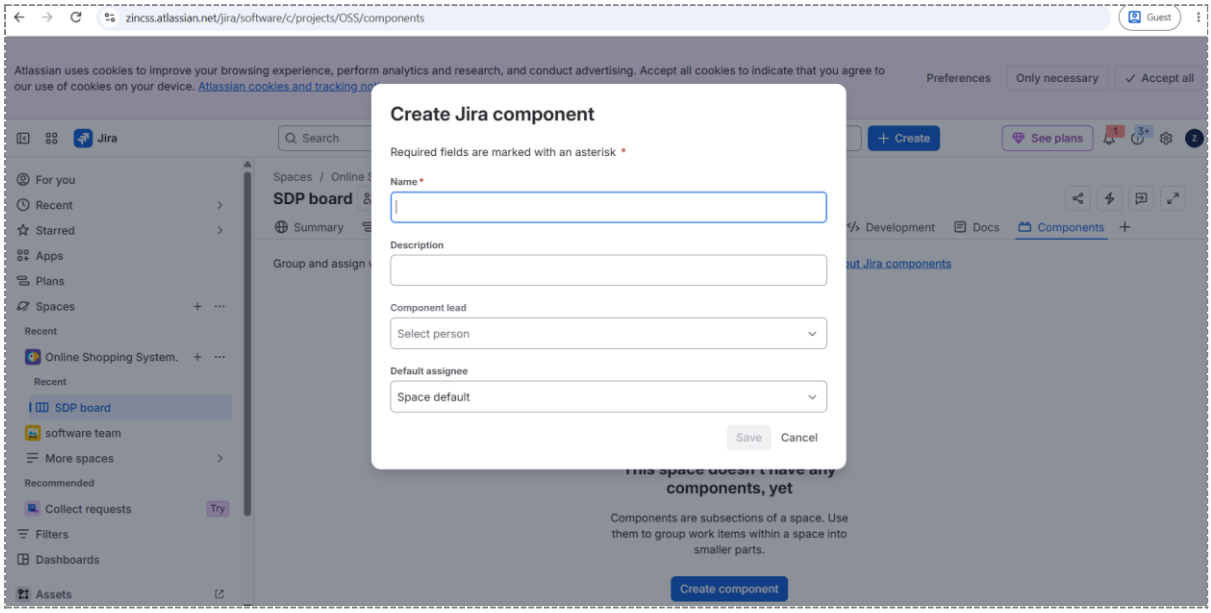


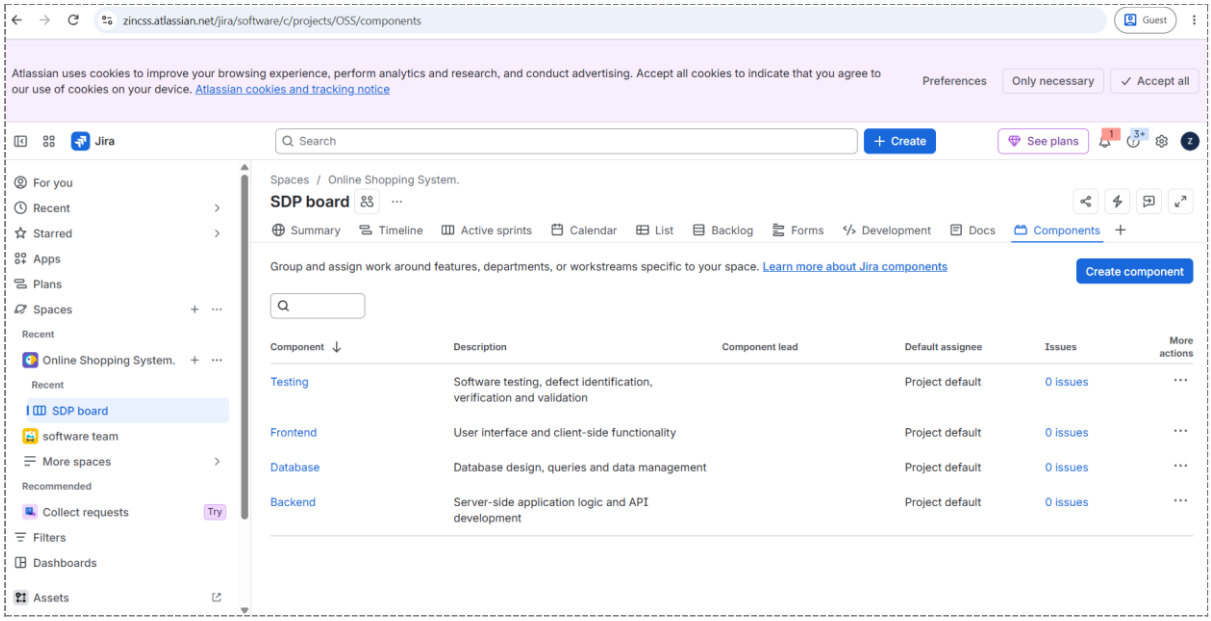
Part B: Create Components

1. Open the project; click Components (if hidden: More actions (•••) → Space settings → Components).
2. Click "Create component". Name: Frontend; Description: "User interface and client-side functionality of the Online Shopping System."; select Component Lead/Default Assignee if required; Save.
3. Create Backend ("Server-side application logic and API development."), Database ("Database design, queries, data management and database integration.") and Testing ("Software testing, defect identification, verification and validation.").
4. Verify all four components on the Components page.

5. Create/open the Story "Customer Login" and in the Component/s field select Frontend.
6. Create a new Bug "Invalid password error not displayed" and set Component/s = Testing.

Output:



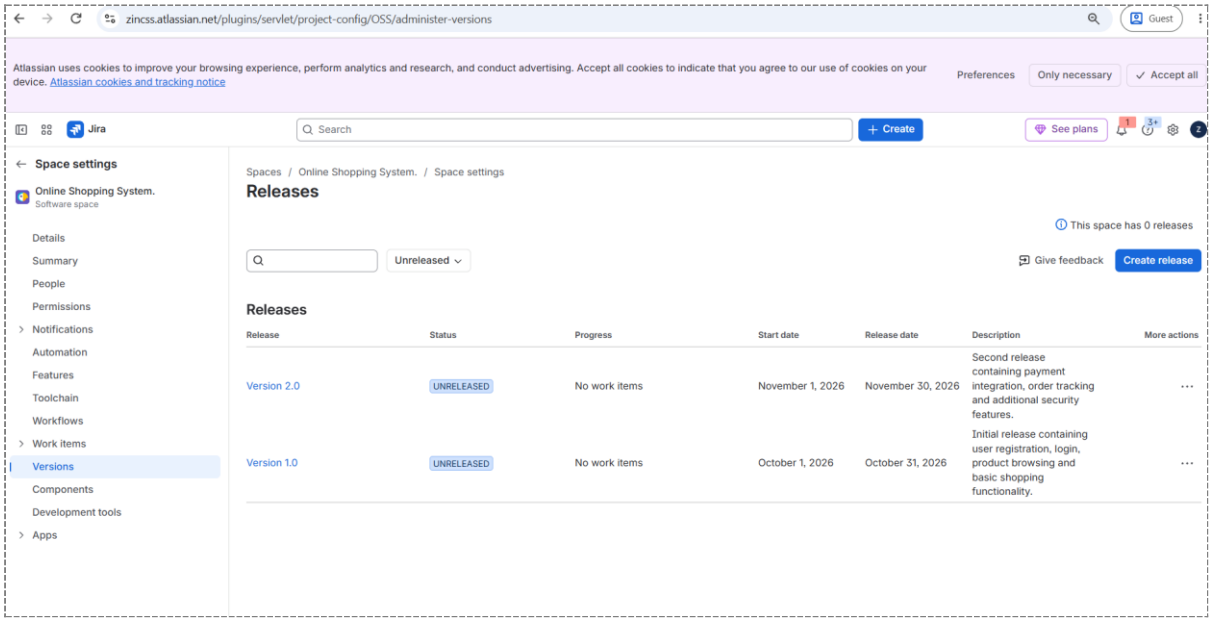


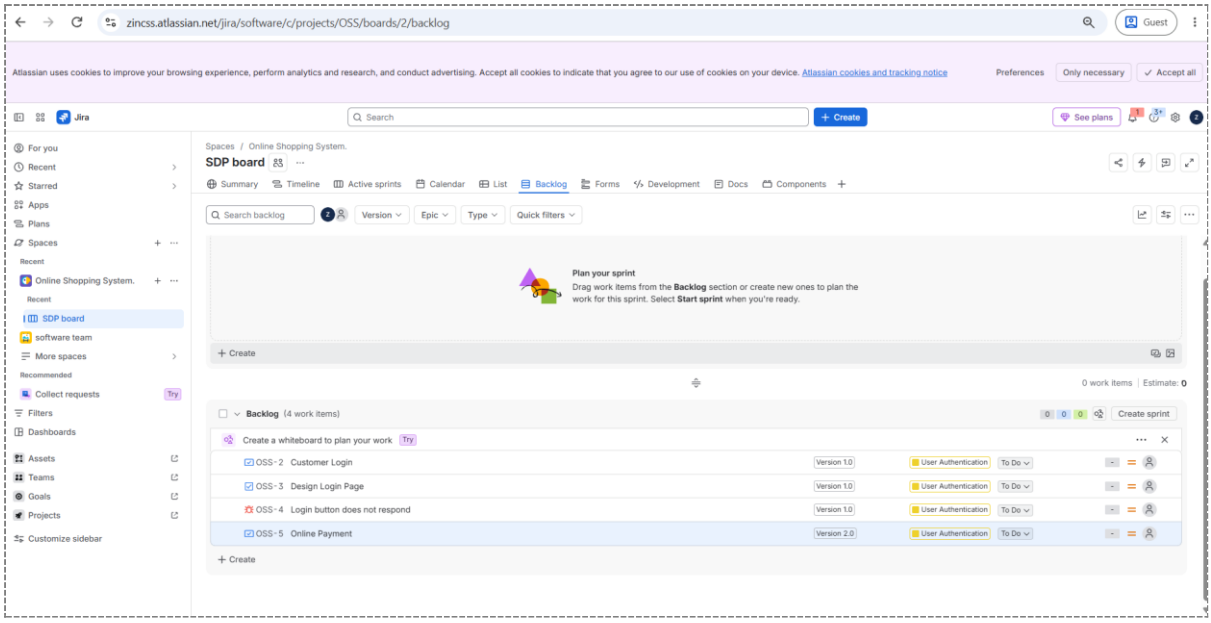
Part C: Create Versions

1. Open Releases in the project navigation (if not visible: ••• → Space settings → Features → enable Releases).

2. Click "Create version". Name: Version 1.0; Start date: 01 October 2026; Release date: 31 October 2026; Description: "Initial release containing user registration, login, product browsing and basic shopping functionality."; Create.
3. Create Version 2.0: Start 01 November 2026, Release 30 November 2026, Description "Second release containing payment integration, order tracking and additional security features."
4. Verify both versions are shown as Unreleased on the Releases page.
5. Open Customer Login → Fix versions field → select Version 1.0 → Save.
6. Set Fix versions = Version 1.0 for the Task "Design Login Page" and the Bug "Login button does not respond".
7. Create Story "Online Payment" and set Fix versions = Version 2.0.

Output:





Result

The required Jira issue types were verified/configured, work items were created successfully, components were created and associated with work items, and software versions/releases were created and assigned to appropriate work items.

PRACTICAL 4

Title: Issue Management

Project: Online Shopping System (Key: OSS) Date: _____ Sign: _____

Aim

To create issues under an Epic, assign issues to users, and configure Priority and Due Date in Jira Cloud.

Requirements

Jira Cloud; project Online Shopping System (OSS); Epic "User Authentication" and team members from earlier practicals.

Final Epic Structure

Type	Summary	Parent
Epic	User Authentication	—
Story	Customer Login	User Authentication
Story	User Registration	User Authentication
Task	Design Login Page	User Authentication
Bug	Login button not responding	User Authentication

Assignee, Priority and Due Date

Work Item	Type	Assignee	Priority	Due Date
Customer Login	Story	User 2	High	30-Sep-2026
User Registration	Story	User 3	Medium	02-Oct-2026
Design Login Page	Task	User 2	—	—
Login button not responding	Bug	User 4	Highest	28-Sep-2026

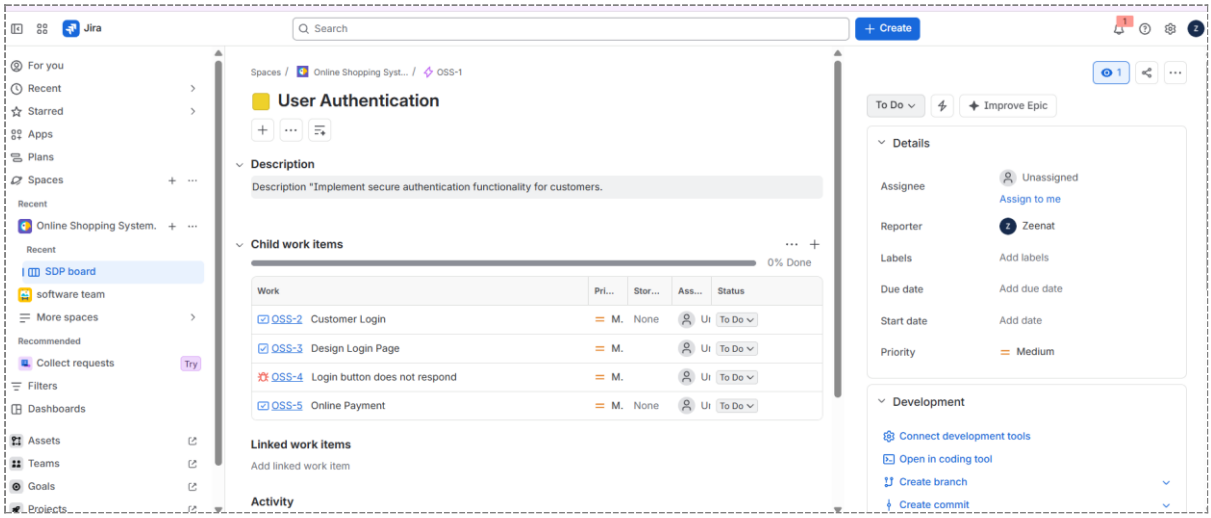
Procedure

Part A: Create Issues Under an Epic

1. Log in, open Spaces/Projects and select Online Shopping System. Open Backlog and locate the Epic "User Authentication" in the Epics panel.
2. Click Create → Work type Story → Summary "Customer Login" → Description "As a registered customer, I want to log in using my email address and password so that I can access my account." → Parent: User Authentication → Create.
3. Open the Story and verify Parent: User Authentication.

4. Create Story "User Registration" (As a new customer, I want to create an account so that I can access the shopping application.) with Parent = User Authentication.
5. Create Task "Design Login Page" (Design the user interface for the customer login page.) with Parent = User Authentication.
6. Create Bug "Login button not responding" (The Login button does not respond after the user enters valid credentials.) with Parent = User Authentication.
7. Open the Epic and verify all its child work items.

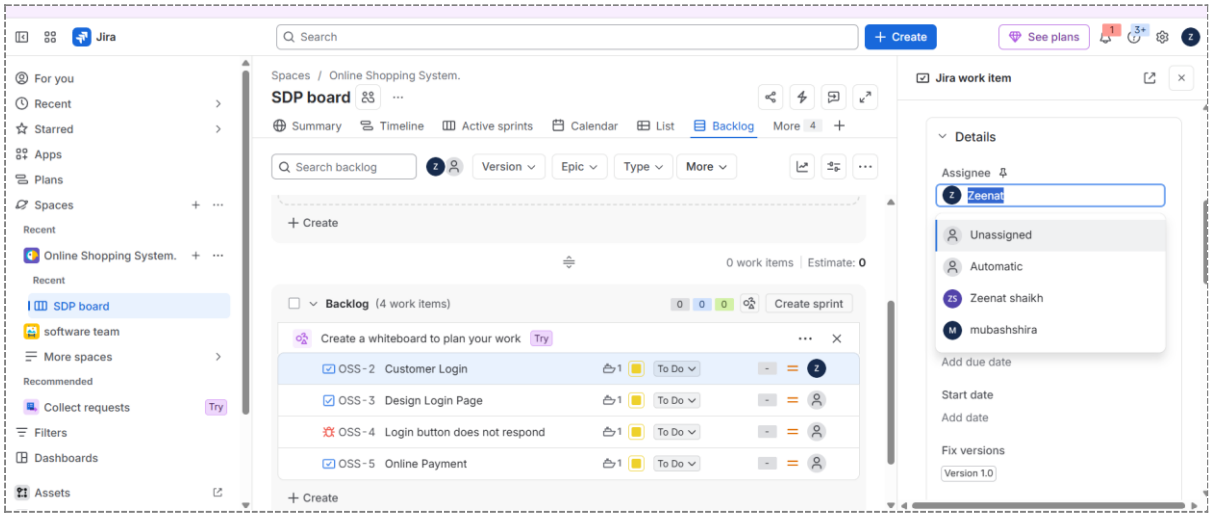
Output:



Part B: Assign Issues to Users

1. Open Customer Login, find the Assignee field (click Edit if needed), search and select User 2, and Save/Update.
2. Set Assignee = User 3 for User Registration.
3. Set Assignee = User 2 for Design Login Page.
4. Set Assignee = User 4 for the Bug "Login button not responding".

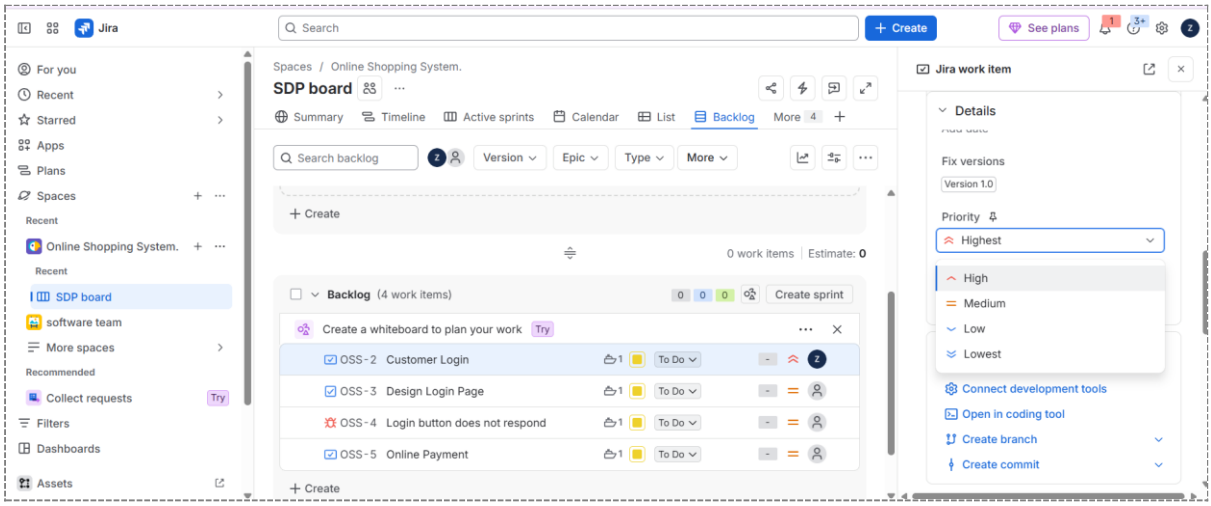
Output:

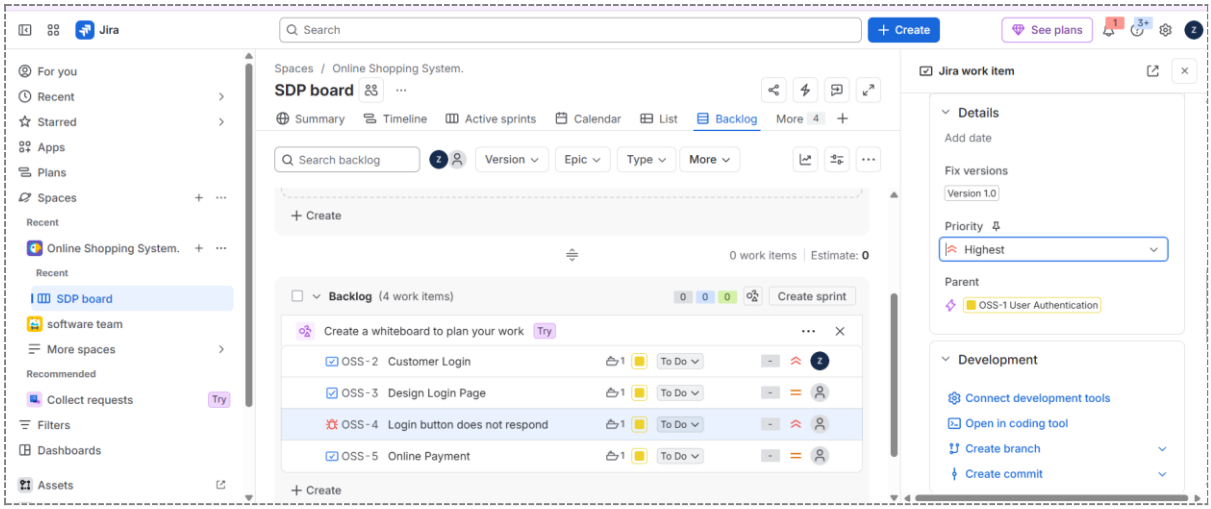


Part C: Set Priority

1. Open Customer Login → Edit → Priority → High → Update.
2. Open User Registration → Priority → Medium → Save.
3. Open the Bug "Login button not responding" → Priority → Highest → Save.

Output:

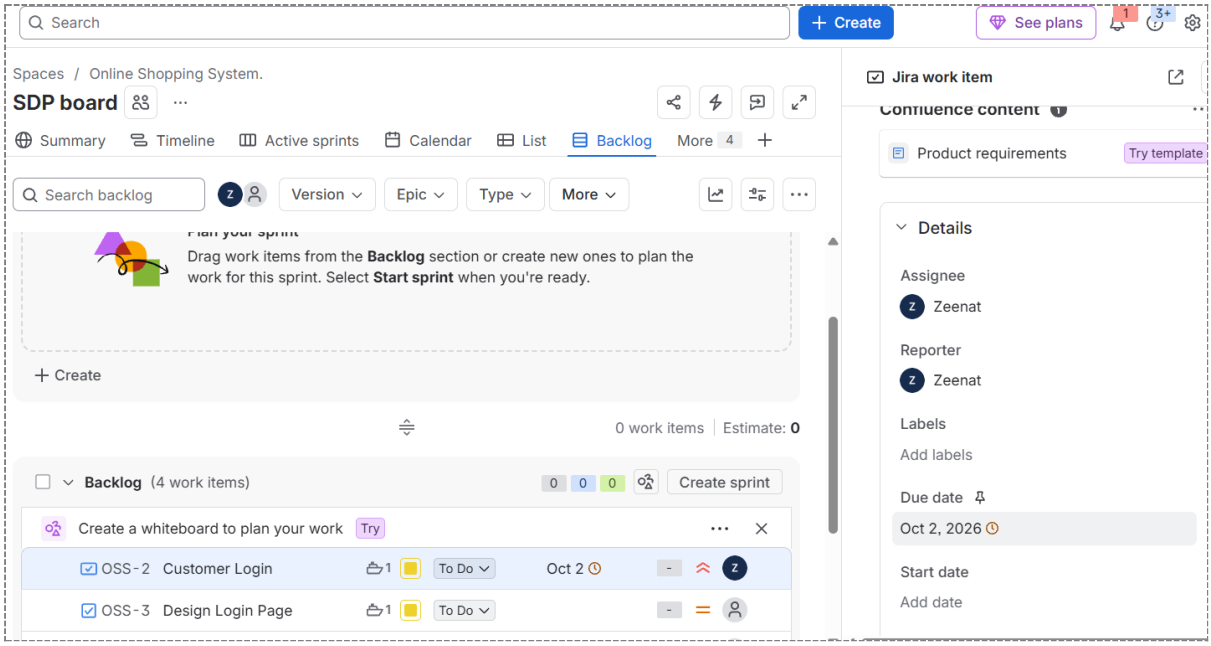




Part D: Set Due Date

1. Open Customer Login → Edit → Due date → 30 September 2026 → Update.
2. Set Due date = 02 October 2026 for User Registration.
3. Set Due date = 28 September 2026 for the Bug.
4. Open each work item and verify Type, Parent, Assignee, Priority and Due date as per the table above.

Output:



Result

Child work items (Stories, Task and Bug) were created under the Epic User Authentication, assigned to team members, and Priority and Due Date were set and verified successfully.

PRACTICAL 5

Title: Workflow Customization

Project: Online Shopping System (Key: OSS) Date: _____ Sign: _____

Aim

To design a custom Jira workflow, add transitions and conditions, publish the workflow, and associate it with a Company-managed project.

Requirements

Jira Cloud with Jira administrator account; project Online Shopping System (OSS) with some existing work items.

Theory

A workflow defines the sequence of statuses through which a work item moves in its lifecycle. Statuses represent the current state (To Do, In Progress, Testing, Done); transitions represent movement between statuses (Start Progress, Pass Testing). A condition controls whether a user may execute a transition; a validator checks that the information needed for the transition is valid/complete; post-functions perform actions after the transition. A workflow is not active until it is published, and it is linked to projects through a workflow scheme: Project → Workflow Scheme → Workflow → Issue types.

Transitions of Software Development Workflow

Transition	From	To	Condition
Start Progress	To Do	In Progress	User is in project role: Developer
Submit for Testing	In Progress	Testing	—
Pass Testing	Testing	Done	Testing/project role condition
Fail Testing	Testing	In Progress	—

Workflow Scheme Mapping

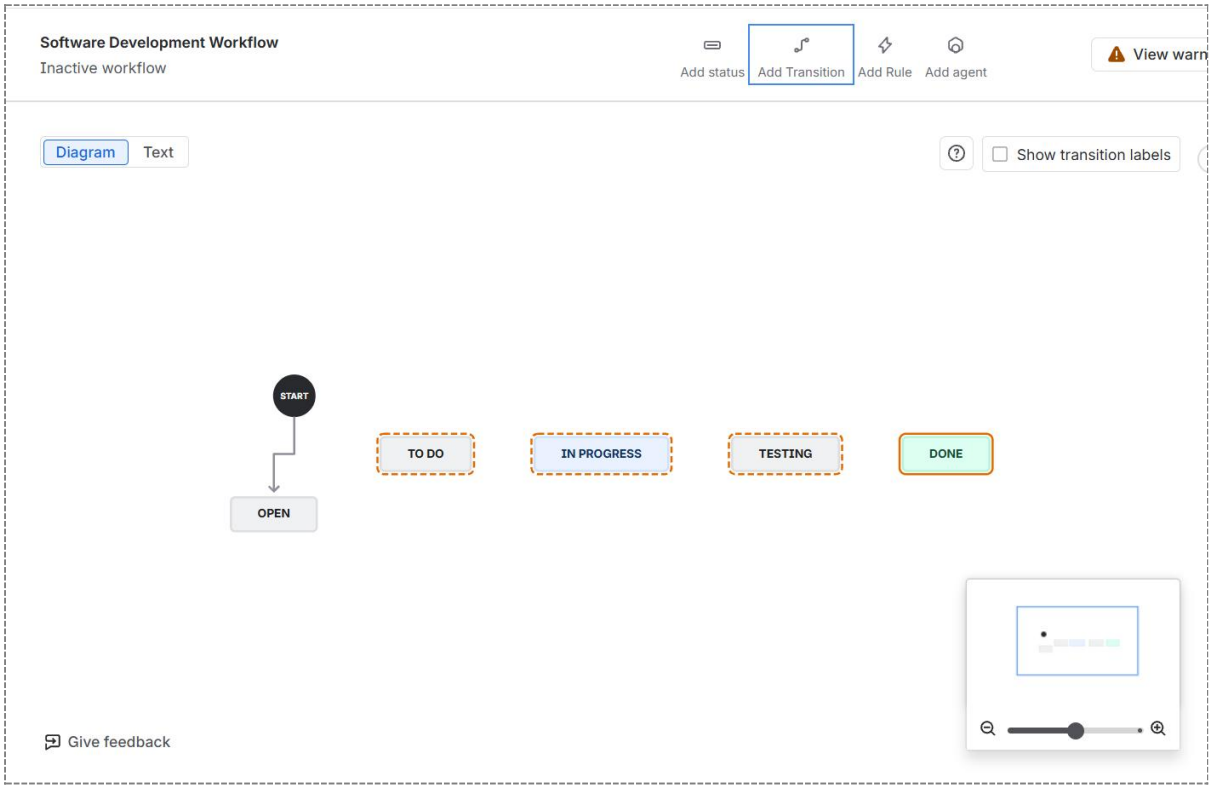
Work Type	Workflow
Story	Software Development Workflow
Task	Software Development Workflow
Bug	Software Development Workflow

Procedure

Part A: Design a Custom Workflow

1. Log in as a Jira administrator. Click Settings (gear) → Work items → Workflows.
2. Select "Add workflow"/"Create workflow".
3. Workflow Name: Software Development Workflow. Description: "Custom workflow for managing software development activities from requirement implementation through testing and completion." Create.
4. Add the four statuses: To Do (work not started), In Progress (being worked on), Testing (development done, being tested), Done (completed and tested). Reuse existing statuses where possible instead of duplicating.

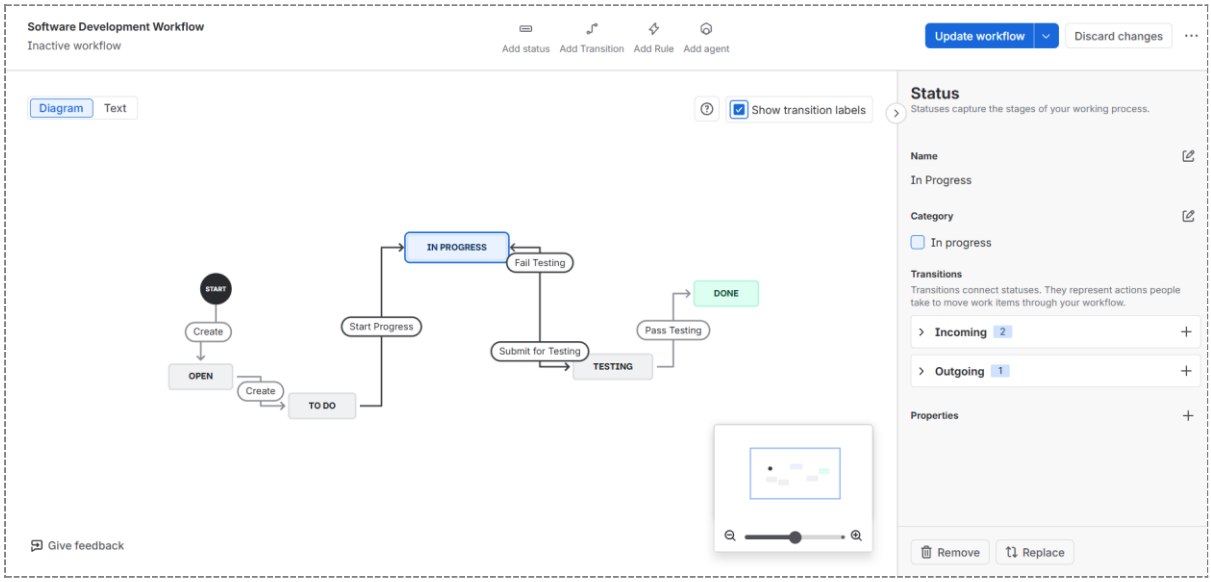
Output:



Part B: Add Transitions

1. Create transition "Start Progress" from To Do to In Progress.
2. Create "Submit for Testing" from In Progress to Testing.
3. Create "Pass Testing" from Testing to Done.
4. Create the rejection transition "Fail Testing" from Testing back to In Progress.
5. Review the complete workflow diagram.

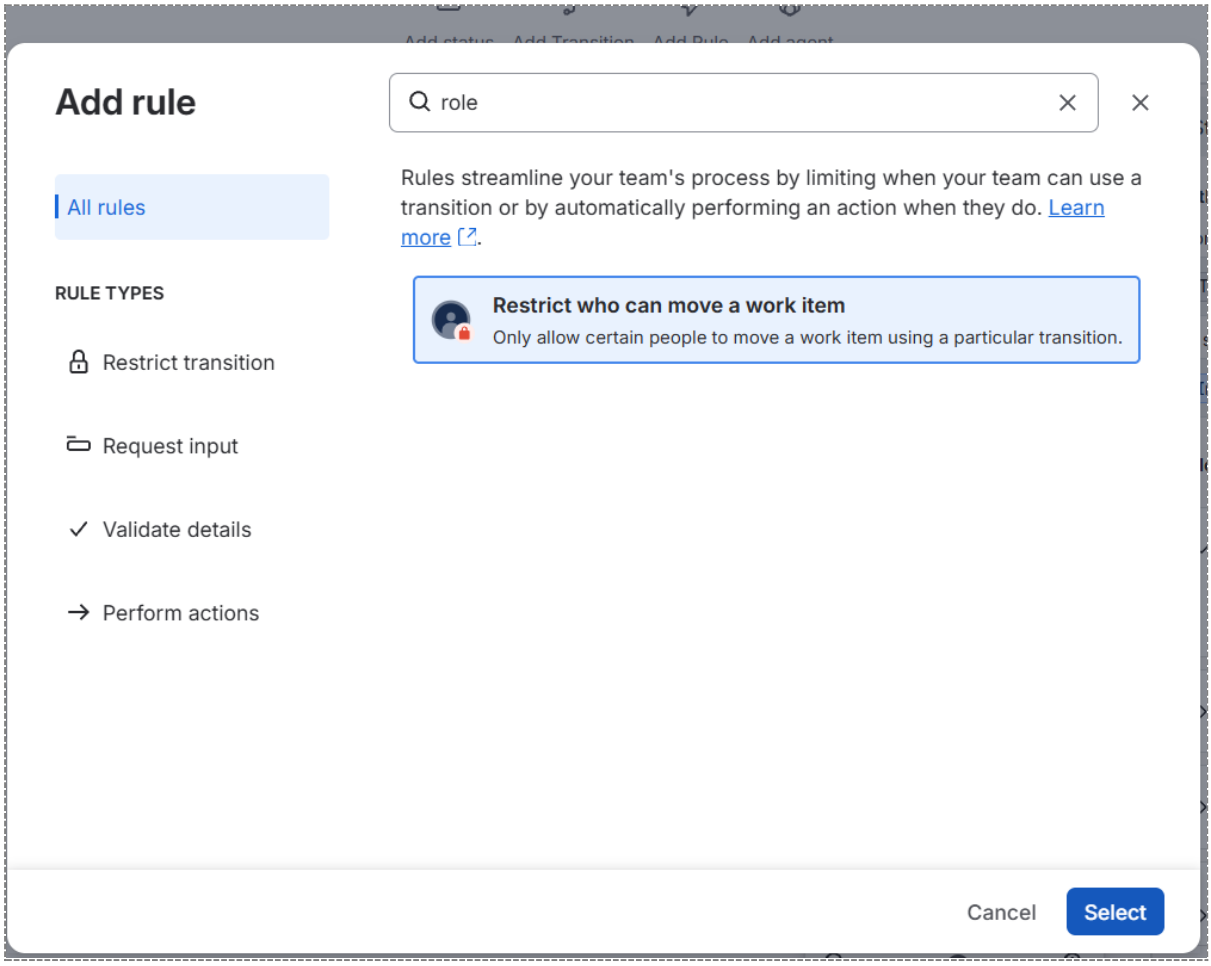
Output:



Part C: Add Conditions

1. Open the "Start Progress" transition → Conditions → Add condition → "User is in project role" → select Developer → Save.
2. Open "Pass Testing" → Conditions → Add condition → choose the appropriate role/group condition (testing role if available) → Save.
3. (Optional) Add a validator to show the difference: a condition decides whether the transition can be performed; a validator checks the required information is valid.

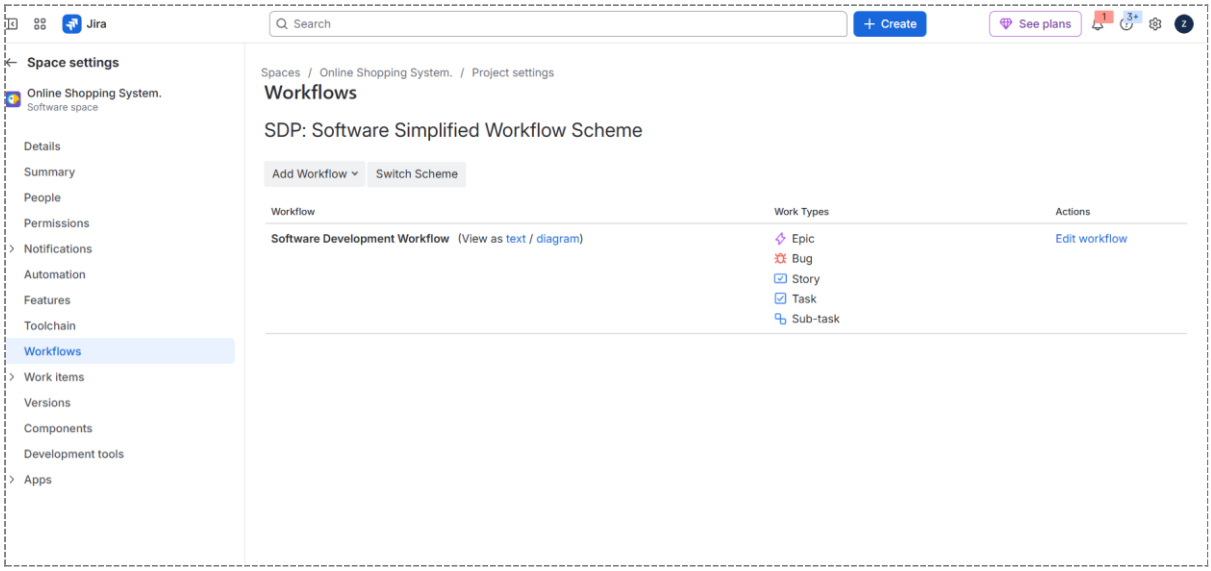
Output:



Part D: Publish the Workflow

1. Save the workflow after adding statuses, transitions and conditions, and review the diagram.
2. Click "Publish draft"/"Publish", review the changes in the confirmation dialog and confirm Publish.

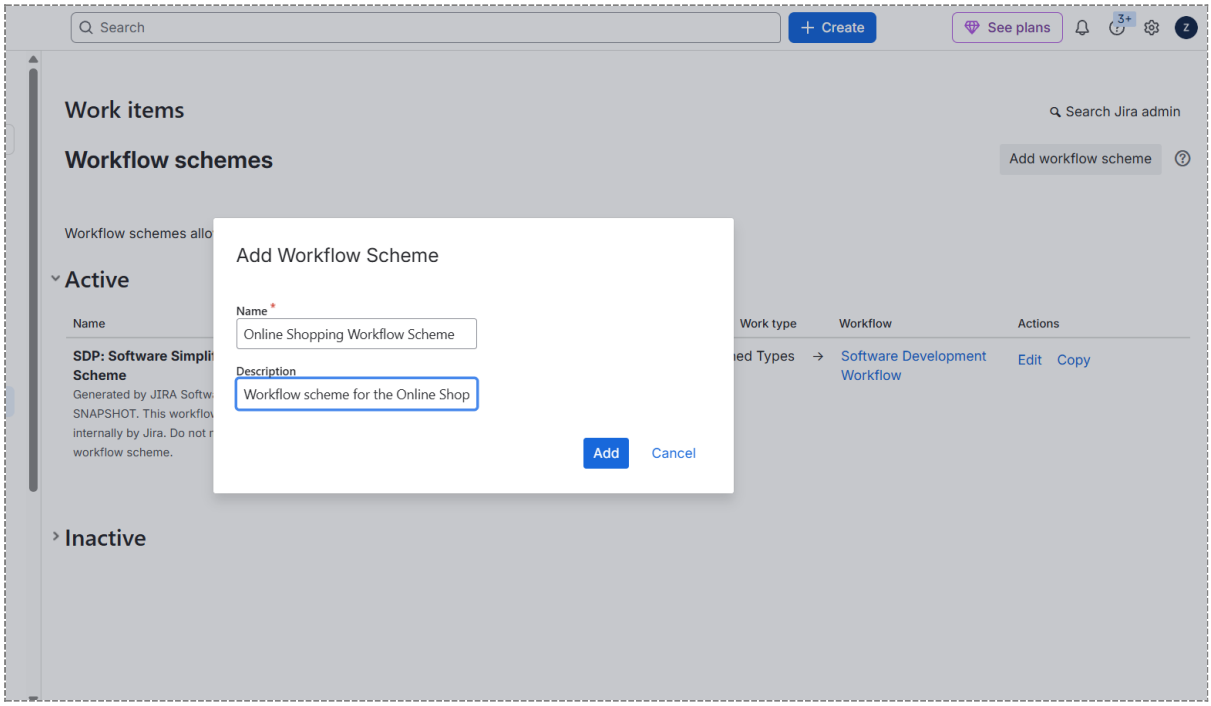
Output:

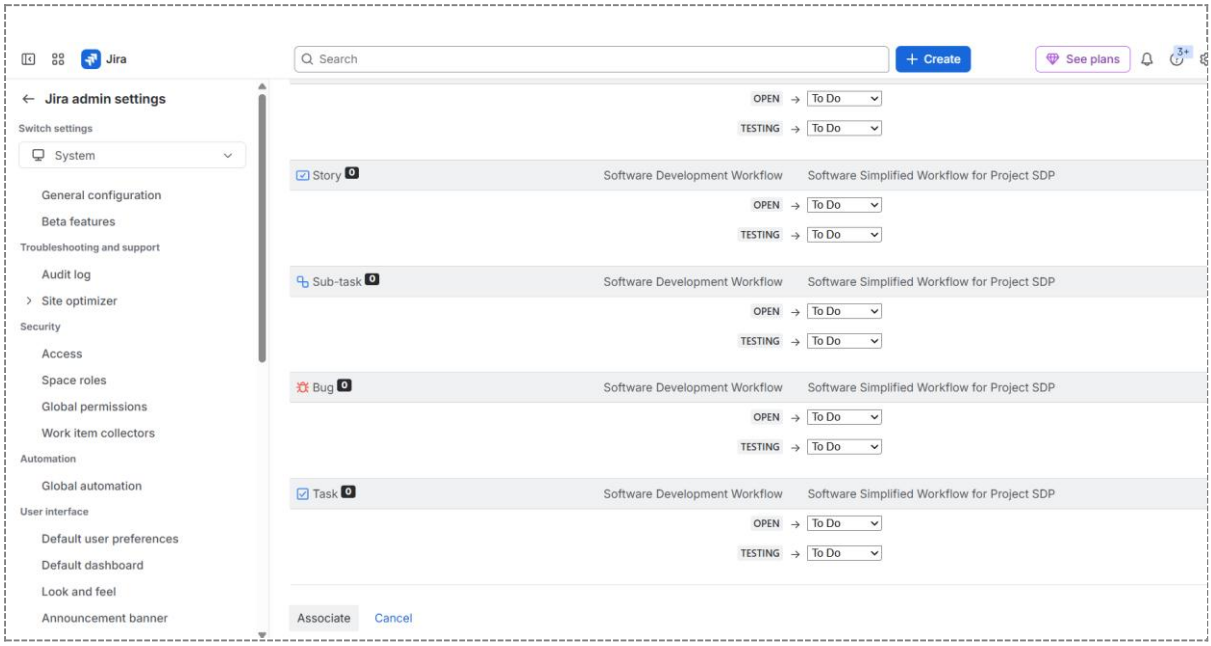


Part E: Associate Workflow with a Project

1. Go to Settings (gear) → Work items → Workflow schemes → "Add workflow scheme".
2. Name: Online Shopping Workflow Scheme; Description: "Workflow scheme for the Online Shopping System project." Save.
3. Associate Story, Task and Bug with Software Development Workflow (and Epic if required).
4. Choose Add project/Associate project and select Online Shopping System; confirm.
5. If Jira asks to migrate existing issues, map statuses (To Do → To Do, In Progress → In Progress, Done → Done) and complete the association.
6. Open a Story in the project and verify the workflow statuses.
7. Test transitions: To Do → In Progress (condition check), In Progress → Testing, Testing → Done, and Testing → In Progress using Fail Testing.

Output:





Result

A custom Software Development workflow was successfully designed with To Do, In Progress, Testing and Done statuses. Required transitions and conditions were configured, the workflow was published, and the workflow scheme was associated with the Online Shopping System project.

PRACTICAL 6

Title: Scrum Board & Sprint Management

Project: Online Shopping System (Key: OSS) Date: _____ Sign: _____

Aim

To create and configure a Scrum board, create and start a sprint, manage work items during the sprint, complete the sprint, and generate/view the Sprint Report in Jira Cloud.

Requirements

Jira Cloud account; Company-managed Scrum project Online Shopping System (OSS); work items from Practical 4 and the custom workflow from Practical 5.

Backlog Work Items

Work Item	Type	Priority
Customer Login	Story	High
User Registration	Story	Medium
Design Login Page	Task	Medium
Login button not responding	Bug	Highest

Board Column Mapping

Column	Status
To Do	To Do
In Progress	In Progress
Testing	Testing
Done	Done

Sprint Details

Item	Value
Sprint Name	Sprint 1 – User Authentication
Sprint Goal	Develop and test the basic user authentication functionality.
Start Date	25 September 2026
End Date	2 October 2026

Work Item Status at Sprint Completion

Work Item	Final Status	Outcome
Customer Login	Done	Completed

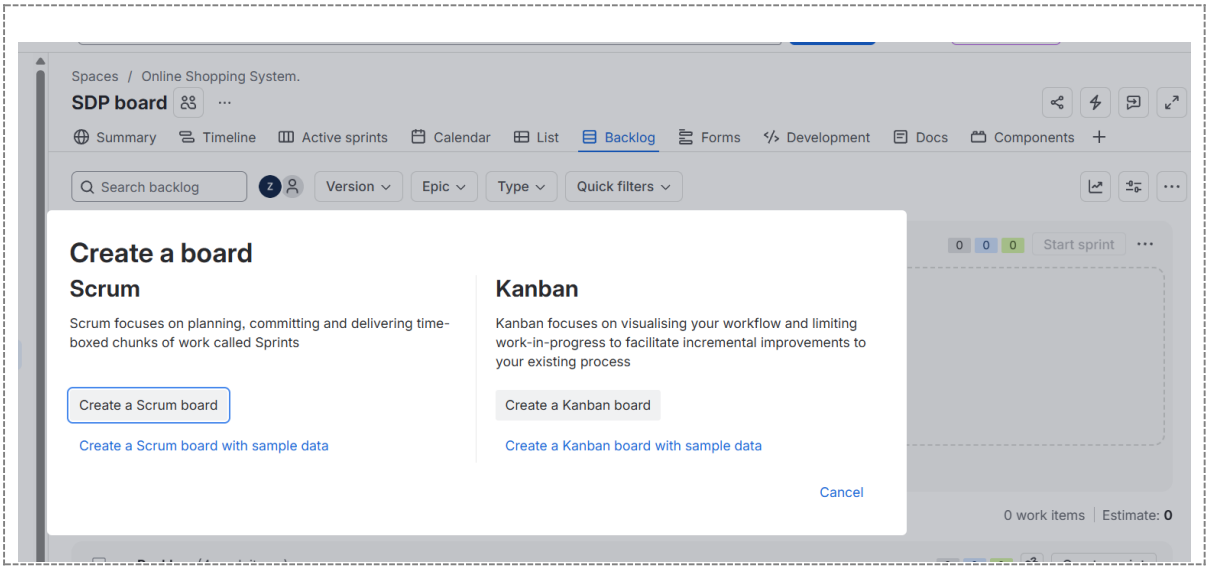
Work Item	Final Status	Outcome
Design Login Page	Done	Completed
User Registration	In Progress	Incomplete – moved to Backlog
Login button not responding	Testing	Incomplete – moved to Backlog

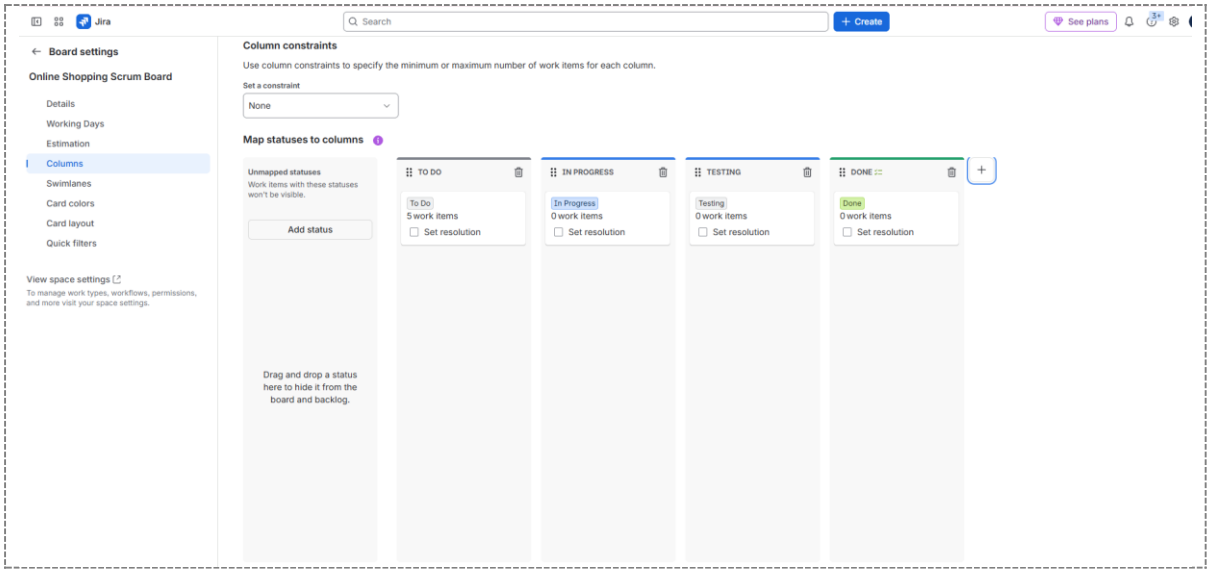
Procedure

Part A: Create and Configure the Scrum Board

1. Log in to Jira Cloud, click Spaces/Projects and select Online Shopping System.
2. From the project navigation open Backlog. The Epic User Authentication and the work items Customer Login, User Registration, Design Login Page and Login button not responding should be visible (reuse items from Practical 4).
3. Check whether the Scrum board already exists: look for Board / Active sprints in the navigation. A Company-managed Scrum project normally provides one automatically; if it exists, use it and do not create another.
4. Only if no board exists: go to Boards → Create board → Scrum board → "Create board from an existing project" → select Online Shopping System → name it "Online Shopping Scrum Board" → Create board.
5. Open Board → Configure (or ••• → Board settings) and select Columns.
6. Check the status mapping and make sure the columns match the workflow created in Practical 5: To Do → To Do, In Progress → In Progress, Testing → Testing, Done → Done. Save the configuration.

Output:

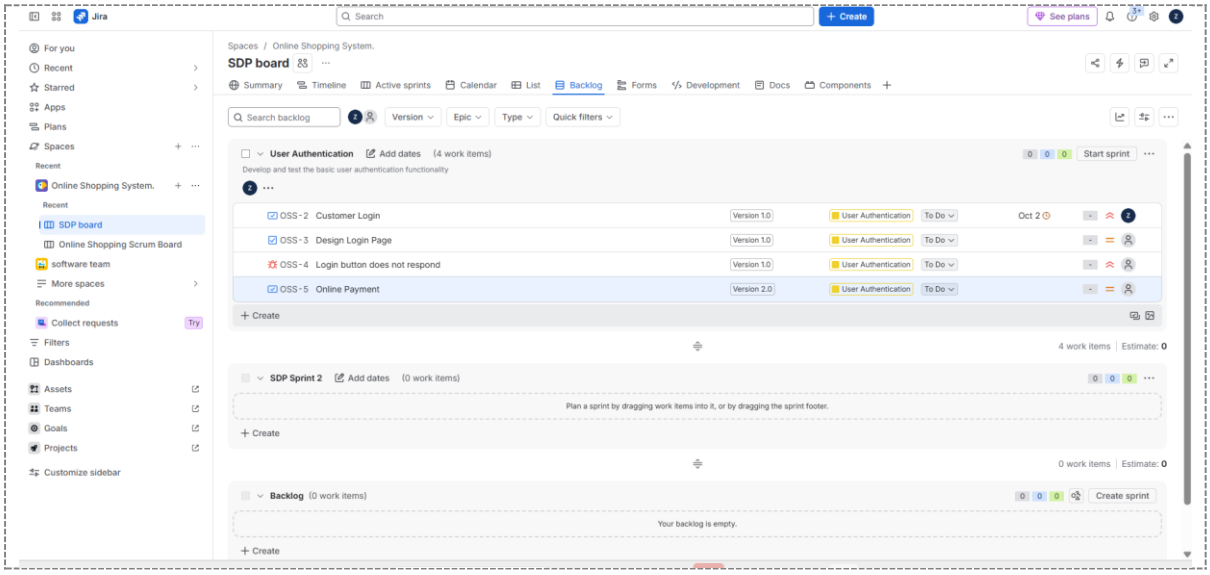


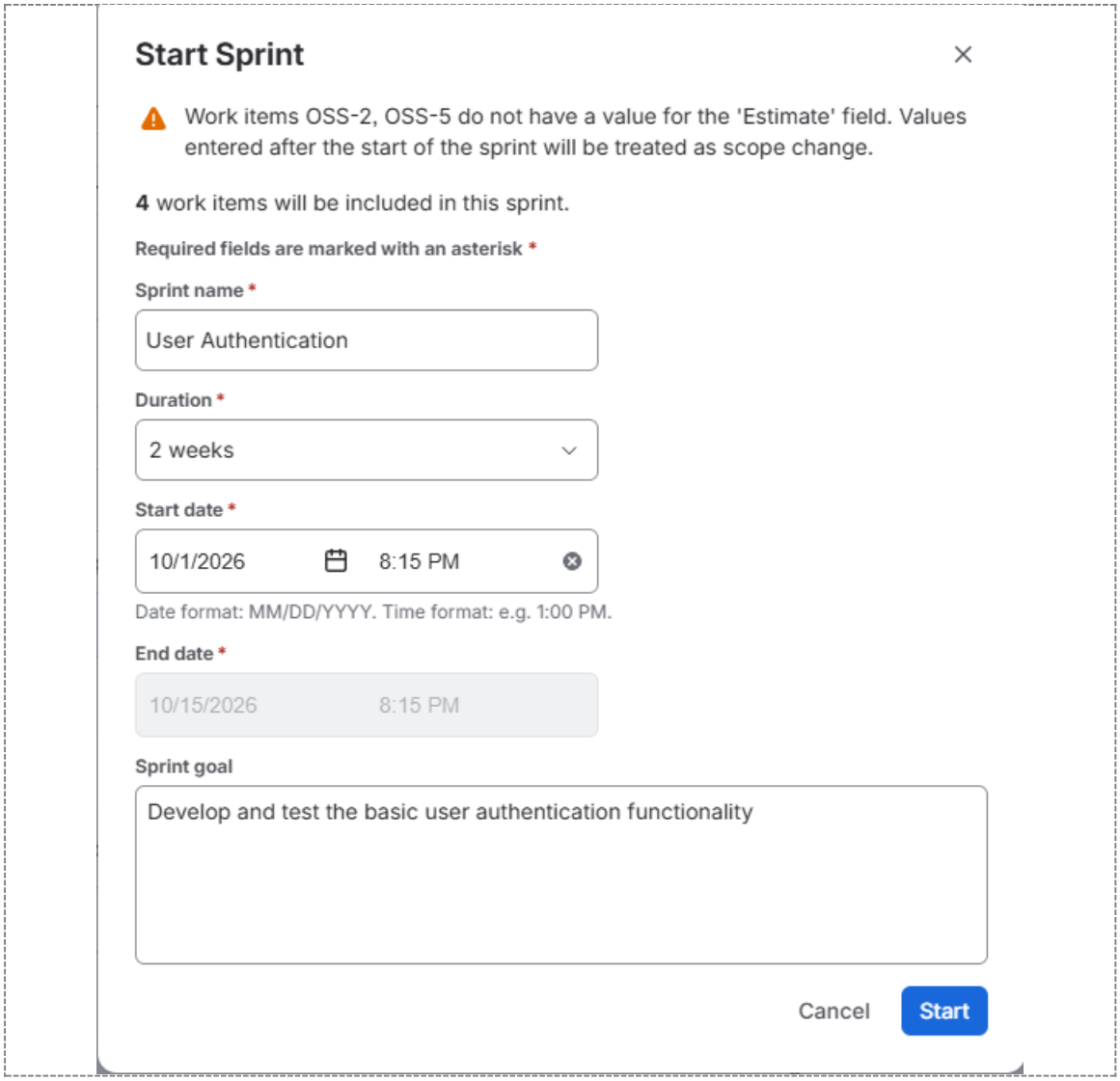


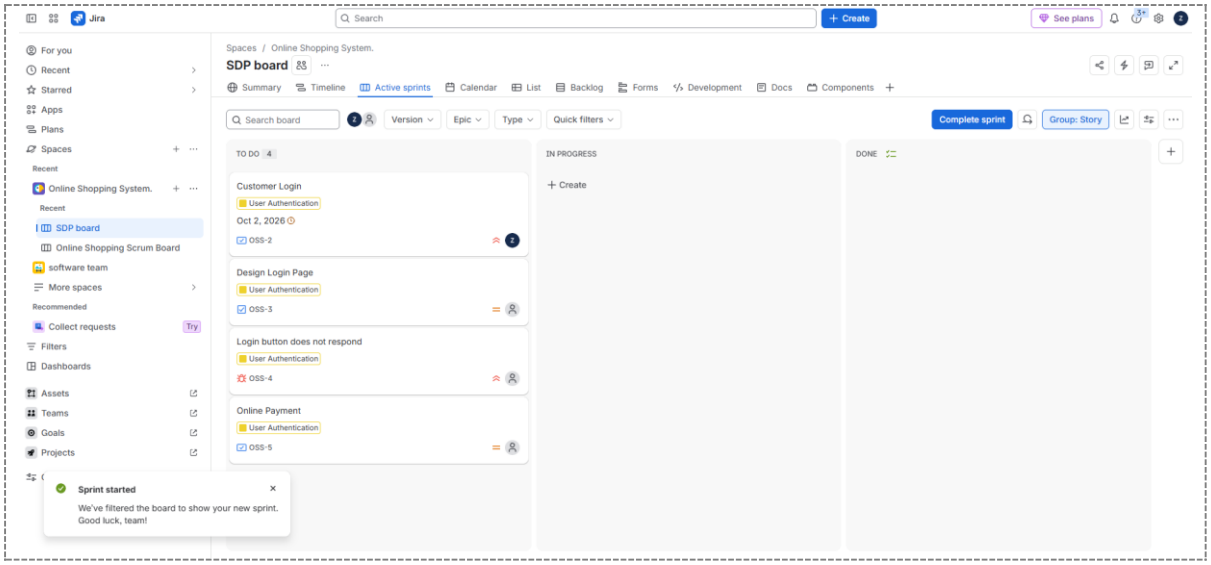
Part B: Create and Start a Sprint

1. Open Online Shopping System → Backlog and view the backlog work items.
2. Click "Create sprint". Jira creates an empty Sprint 1.
3. Rename the sprint to "Sprint 1 – User Authentication" and enter the Sprint Goal: "Develop and test the basic user authentication functionality."
4. Add work items to the sprint by dragging them from the Backlog (or using the ••• menu and choosing the sprint): Customer Login, User Registration, Design Login Page and Login button not responding.
5. Set the sprint Start date (25 September 2026) and End date (2 October 2026).
6. Click "Start sprint", check the sprint name, dates and goal in the dialog, and click Start. The sprint is now active.
7. Open Active sprints to view the sprint board.
8. Demonstrate the workflow: move Customer Login from To Do → In Progress (Start Progress) → Testing (Submit for Testing) → Done (Pass Testing).
9. Update the other items: User Registration → In Progress, Design Login Page → Done, Login button not responding → Testing.

Output:


Jira
Search
Create
See plans
For you
Recent
Starred
Apps
Plans
Spaces
Online Shopping System.
SDP board
Online Shopping Scrum Board
software team
More spaces
Collect requests
Filters
Dashboards
Assets
Teams
Goals
Projects
Customize sidebar
Spaces / Online Shopping System.
SDP board
Summary
Timeline
Active sprints
Calendar
List
Backlog
Forms
Development
Docs
Components
Search backlog
Version
Epic
Type
Quick filters
User Authentication
Add dates
(4 work items)
Start sprint
Develop and test the basic user authentication functionality
OSS-2 Customer Login
OSS-3 Design Login Page
OSS-4 Login button does not respond
OSS-5 Online Payment
Version 1.0
Version 2.0
User Authentication
To Do
Oct 2
Create
4 work items | Estimate: 0
SDP Sprint 2
Add dates
(0 work items)
Plan a sprint by dragging work items into it, or by dragging the sprint footer.
Create
0 work items | Estimate: 0
Backlog
(0 work items)
Create sprint
Your backlog is empty.
Create

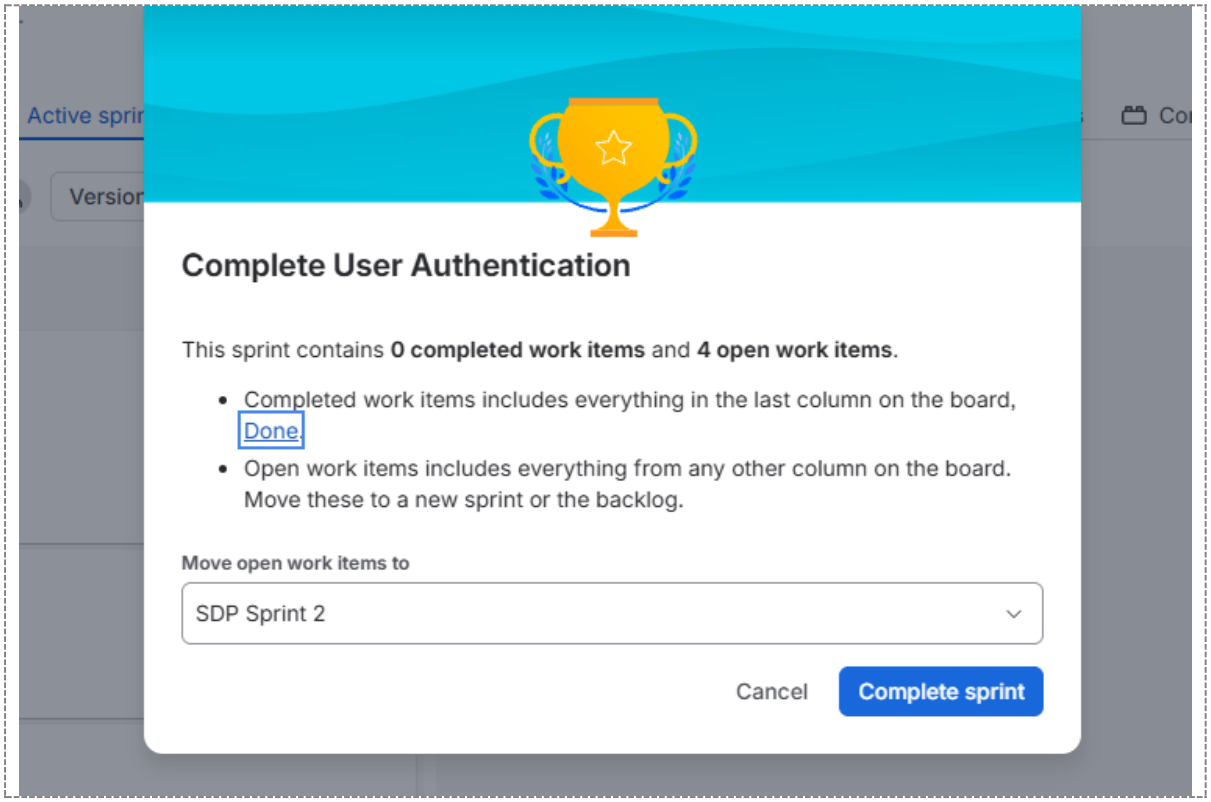

Start Sprint
Work items OSS-2, OSS-5 do not have a value for the 'Estimate' field. Values entered after the start of the sprint will be treated as scope change.
4 work items will be included in this sprint.
Required fields are marked with an asterisk *
Sprint name *
User Authentication
Duration *
2 weeks
Start date *
10/1/2026
8:15 PM
Date format: MM/DD/YYYY. Time format: e.g. 1:00 PM.
End date *
10/15/2026
8:15 PM
Sprint goal
Develop and test the basic user authentication functionality
Cancel
Start

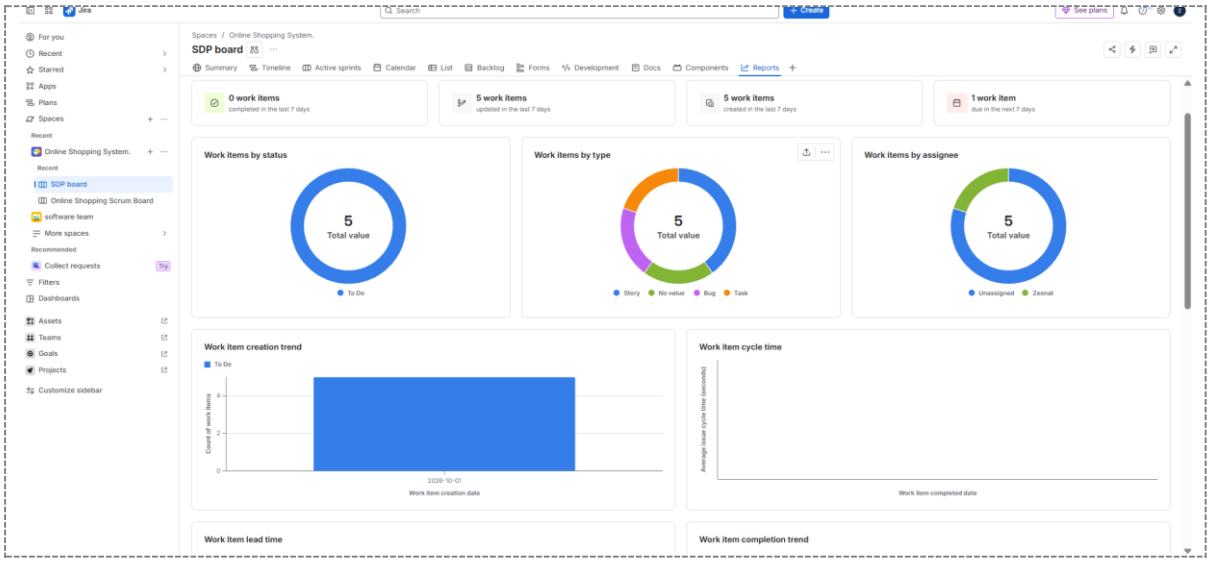


Part C: Complete the Sprint and Generate Sprint Report

1. Go to Active sprints and click "Complete sprint".
2. Review the sprint completion screen: completed work items are Customer Login and Design Login Page; incomplete work items are User Registration and Login button not responding.
3. For the incomplete items select "Move to Backlog" and click "Complete sprint". The sprint is now closed.
4. Go to Reports and select Sprint Report.
5. Choose Sprint 1 – User Authentication from the sprint list.
6. Review the report: completed work, incomplete work, work added during the sprint, sprint progress and story points (if used).

Output:





Result

A Scrum board was successfully configured, a sprint was created and started, work items were managed through the Scrum workflow (To Do → In Progress → Testing → Done), the sprint was completed, and the Sprint Report was generated and reviewed.